



Universitatea
Transilvania
din Brașov

FACULTATEA DE ȘTIINȚE ECONOMICE
ȘI ADMINISTRAREA AFACERILOR

LUCRARE DE DISERTAȚIE

Conducător științific:

Conferențiar dr. BÎLDEA Teodor-Ștefan

Absolvent:

Bolboacă Mara

BRAȘOV, 2026

Universitatea Transilvania din Braşov
Facultatea de Ştiinţe Economice şi Administrarea Afacerilor
Program de studii: Sisteme Informatice Integrate în Afaceri

Optimizarea stocurilor în sectorul retail folosind metode predictive

Conducător ştiinţific:

Conferenţiar dr. BÎLDEA Teodor-Ştefan

Absolvent:

Bolboacă Mara

BRAŞOV, 2026

CUPRINS

Contents

CAPITOLUL 1 - Introducere.....	1
1.1 Contextul general	1
1.2 Obiectivele cercetării.....	3
1.3 Contribuția lucrării	3
Capitolul 2. Stadiul actual al cercetărilor și aplicațiilor în domeniu (5–7 pagini)	4
2.1. Rolul sistemelor informatice integrate (ERP) în gestiunea și optimizarea stocurilor.....	4
2.2. Utilizarea Data Science și Machine Learning în prognoza cererii.....	5
2.2.2. Tipuri de abordări în Machine Learning utilizate în prognoza cererii	6
2.2.3. Serii temporale în retail.....	7
2.2.4. Metode statistice vs Machine Learning	8
2.2.5. Modele utilizate în literatura de specialitate	10
2.3. Aplicații practice și rezultate raportate în literatura de specialitate	16
2.4. Limitări identificate în literatura de specialitate	Error! Bookmark not defined.
Capitolul 3. Metodologia de cercetare (1–3 pagini).....	18
3.1. Tipul cercetării	18
3.2. Datele utilizate	18
3.3. Etapele cercetării.....	19
3.4. Instrumente și tehnologii utilizate.....	20
3.5. Variabile și indicatori analizați.....	21
Capitolul 4. Rezultate și aplicația dezvoltată (25–30 pagini).....	22
4.1 Generarea setului de date si pre-processing.....	22
4.2. Analiza exploratorie a datelor	24
4.3. Descompunerea seriilor temporale.....	27
4.3. Modele de prognoză utilizate.....	28
4.5. Evaluarea performanței modelelor.....	38
4.6. Aplicația de tip dashboard pentru vizualizare.....	38
Capitolul 5. Discuții și analize (aprox. 10 pagini).....	40

5.1. Interpretarea rezultatelor.....	40
5.2. Implicații pentru compania analizată	40
5.3. Implicații pentru domeniul sistemelor informatice integrate	40
CAPITOLUL 6 – CONCLUZII, PROPUNERI ȘI IMPLICAȚII PRACTICE.....	40
Limitări, concluzii și recomandări practice (<i>3–5 pagini</i>).....	40
Z.1. Concluzii.....	Error! Bookmark not defined.
Z.2. Propuneri.....	Error! Bookmark not defined.
Z.3. Implicații practice.....	Error! Bookmark not defined.
BIBLIOGRAFIE.....	44

CAPITOLUL 1 - Introducere

1.1 Contextul general

În contextul dezvoltării rapide a soluțiilor bazate pe inteligența artificială în tot mai multe domenii ale vieții economice și sociale, pentru a rămâne competitive și relevante pe piață, majoritatea companiilor își aliniază activitatea la noile cerințe tehnologice.

Dincolo de trend, inteligența artificială aduce îmbunătățiri incontestabile în activitatea organizațiilor prin automatizarea sarcinilor repetitive, reducerea timpilor de procesare, creșterea acurateții analizelor și îmbunătățirea capacității de prognoză.

„Cea mai bună cale de a prezice viitorul este să-l creezi.” – Peter Drucker¹

Prezenta lucrare se concentrează asupra metodelor de predicție construite cu inteligența artificială, folosite pentru a determina viitoare informații despre afaceri bazându-se pe tipare istorice. Valoarea principală a acestor metode, nu este „prezicerea viitorului”, ci identificarea și conștientizarea diferitelor scenarii posibile, fundamentate pe tendințe

¹ StartCo - Newsletter pentru antreprenori

istorice reale, precum și dezvoltarea abilităților de adaptare în fața neprevăzutului. În acest sens, metodele predictive bazate pe inteligență artificială nu vizează exclusiv anticiparea unor valori viitoare, ci oferă un cadru analitic pentru înțelegerea și gestionarea incertitudinii. Tema este fundamentată pe un studiu de caz real, realizat pe baza datelor operaționale ale unei organizații comerciale, în care procesele de aprovizionare, vânzare și gestiune a stocurilor sunt susținute de un sistem ERP conectat la o bază de date relațională.

Problema cercetată

În urma analizei capabilităților și nevoilor companiei în care lucrez, una dintre provocările întâlnite de clienți o reprezintă planificarea eficientă a stocurilor, provocare discutată în cadrul sistemelor informatice integrate din retail în condițiile unui comportament de consum variabil și al unor termene de livrare dependente de furnizori. În practica organizației analizate, deciziile de aprovizionare se bazează în principal pe indicatori simpli, precum vânzarea medie zilnică, utilizată pentru estimarea cantităților comandate și a perioadei de acoperire a stocului.

Deși această abordare oferă un nivel de control operațional, ea nu integrează în mod explicit dinamica cererii și nu permite o anticipare suficient de precisă a situațiilor de suprastoc sau de lipsă de stoc. Astfel, comenzile sunt planificate pe baza unui număr fix de zile de aprovizionare, stabilit fie la nivel de furnizor, fie printr-o variabilă generală, fără a ține cont de evoluția reală a cererii sau de variațiile istorice ale vânzărilor.

În plus, lipsa unor estimări pe termen mediu sau a unei integrări a evenimentelor sezoniere conduce la decizii reactive, cu impact direct asupra performanței lanțului de aprovizionare. Probleme suplimentare apar și din calitatea datelor operaționale, cum ar fi situațiile de stoc negativ, erorile de recepție sau scanare și perioadele de „out of stock”, care pot distorsiona analiza dacă nu sunt tratate corespunzător.

Importanța problemei

Gestionarea ineficientă a stocurilor are un impact semnificativ asupra performanței financiare și operaționale a organizațiilor din retail. Suprastocarea conduce la blocarea capitalului, creșterea costurilor de depozitare și riscul de depreciere a produselor, în timp ce situațiile de „out of stock” generează pierderi de vânzări, scăderea satisfacției clienților și afectarea imaginii comerciale.

În contextul analizat, optimizarea stocurilor nu reprezintă doar o problemă de reducere a costurilor, ci un element strategic pentru susținerea competitivității. Integrarea prognozelor generate prin metode predictive în procesele ERP permite trecerea de la o abordare bazată

pe vânzări istorice simple la una orientată către cererea prognozată, corelată cu timpii reali de aprovizionare și cu nivelurile minime de stoc dorite.

1.2 Obiectivele cercetării

Obiectivul principal al lucrării este analizarea și evaluarea modului în care metodele predictive bazate pe inteligență artificială pot fi utilizate pentru optimizarea proceselor de gestiune a stocurilor, pornind de la datele operaționale extrase dintr-un sistem informatic ERP și valorificate ulterior prin instrumente de Business Intelligence. În acest context, sistemul ERP este utilizat ca sursă de date, fără a fi modificat din punct de vedere funcțional, iar accentul este pus pe analiza și interpretarea rezultatelor predictive în stratul analitic.

Lucrarea urmărește să evidențieze impactul utilizării prognozelor de cerere generate pe baza datelor istorice asupra procesului de planificare a comenzilor către furnizori, comparativ cu abordarea tradițională bazată pe vânzarea medie zilnică. De asemenea, este analizat un flux automatizat de prognoză realizat în Python, care prelucrează datele operaționale și furnizează rezultate structurate pentru analiză și vizualizare în instrumente de Business Intelligence. Prin această abordare, se urmărește identificarea diferențelor dintre cele două metode de fundamentare a deciziilor de aprovizionare, în special în ceea ce privește nivelurile de stoc rezultate și capacitatea organizației de a reduce situațiile de suprastoc sau lipsă de stoc.

1.3 Contribuția lucrării

Contribuția acestei lucrări constă în propunerea și analizarea unei soluții analitice pentru optimizarea gestiunii stocurilor, bazată pe utilizarea metodelor predictive de inteligență artificială aplicate pe date reale dintr-un sistem ERP. Lucrarea evidențiază modul în care datele operaționale pot fi valorificate într-un strat analitic independent, fără a interveni asupra logicii funcționale a sistemului ERP, prin intermediul unui flux automatizat de prelucrare și prognoză realizat în Python.

Prin studiul de caz realizat, lucrarea contribuie la reducerea decalajului dintre modelele teoretice de prognoză prezentate în literatura de specialitate și aplicabilitatea lor practică în organizații comerciale. Totodată, sunt evidențiate bune practici privind utilizarea instrumentelor de Business Intelligence ca suport decizional în procesele de optimizare a stocurilor, oferind un cadru replicabil pentru organizații care doresc să îmbunătățească performanța operațională prin analitică predictivă.

Capitolul 2. Stadiul actual al cercetărilor și aplicațiilor în domeniu

2.1. Rolul sistemelor informatice integrate (ERP) în gestiunea și optimizarea stocurilor

Sisteme ERP – Definiție și rol în retail

Sistemele pentru planificarea resurselor întreprinderii (Enterprise Resource Planning – ERP) reprezintă platforme software integrate care centralizează și automatizează procesele de afaceri ale unei organizații, de la aprovizionare și vânzări până la gestiunea stocurilor și raportare financiară. Aceste sisteme au evoluat de la simple instrumente de management al producției către soluții comprehensive care integrează întreaga activitate operațională într-o singură bază de date centralizată².

Sistemele ERP permit monitorizarea continuă a nivelurilor de stoc, a mișcărilor de intrare/ieșire și a valorii inventarului pe magazine, depozite sau locații multiple. Această vizibilitate elimină problemele generate de datele fragmentate și permite identificarea rapidă a anomaliilor.³

Limitări și nevoia de instrumente complementare

Conform EazyStock (2025), multe module de gestiune a stocurilor din ERP oferă funcționalități de prognoză a cererii, de obicei bazate pe calcule simple de medie mobilă liniară, dar aceste metode nu pot lua în considerare cauzele tipice ale volatilității cererii.

Spre deosebire de metodele tradiționale, un sistem de optimizare a stocurilor va lua în considerare ciclul de viață al produsului, sezonabilitatea, tendințele și informațiile despre promoții – aspecte pe care modulele standard ERP nu le gestionează eficient. Această abordare complexă este ilustrată în figura de mai jos (fig. 1.), care evidențiază modul în care prognozele avansate integrează atât prognoza de bază (base forecast), cât și factori calitativi suplimentari precum tendințele (trends), sezonabilitatea (seasonality factors) și alte variabile contextuale (qualitative input).⁴

² <https://www.oracle.com/erp/what-is-erp/>

³ SAP, 2024 - <https://www.sap.com/products/erp/what-is-erp.html>

⁴ <https://www.eazystock.com/blog/erp-inventory-management-how-advanced-is-your-erp/>

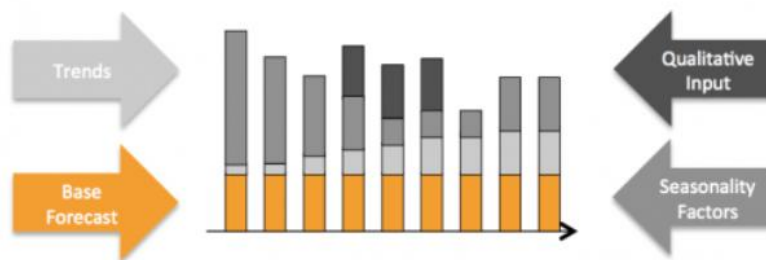


Fig. 1 Factorii care afectează prognoza cererii

2.2. Utilizarea Data Science și Machine Learning în prognoza cererii

Data Science permite analizarea volumelor mari de date istorice provenite din sisteme informatice integrate și identificarea unor tipare complexe de consum, oferind o bază mai solidă pentru estimările predictive comparativ cu metodele tradiționale. În special în sectorul de retail, caracterizat prin sezonabilitate, promoții frecvente și variații ale comportamentului consumatorilor, aceste tehnici contribuie la o mai bună înțelegere a dinamicii cererii și la fundamentarea deciziilor operaționale.⁵

Un aspect evidențiat în cercetarile recente este faptul că valoarea reală a inteligenței artificiale în prognoza cererii nu constă exclusiv în creșterea acurateții estimărilor, ci în capacitatea de a gestiona variabilitatea și incertitudinea inerente proceselor operaționale. Modelele predictive sunt utilizate pentru a analiza scenarii alternative și pentru a evalua impactul deciziilor asupra fluxului de marfă și asupra capitalului blocat în stocuri, mutând accentul de la o prognoză punctuală către înțelegerea comportamentului sistemului în ansamblu.⁶

Ce este Machine Learning?

Machine Learning este ramura inteligenței artificiale ce cuprinde dezvoltarea de algoritmi capabili să învețe automat din date și să își îmbunătățească performanța fără să fie programați detaliat pentru fiecare situație. Prin identificarea tiparelor și a relațiilor din seturi mari de date, aceste algoritmi pot realiza predicții sau pot sprijini luarea deciziilor în contexte complexe, precum prognoza cererii, analiza comportamentului consumatorilor sau optimizarea proceselor operaționale. Machine Learning este utilizat pe scară largă în mediile

⁵ <https://www.ibm.com/think/topics/ai-demand-forecasting>

⁶ <https://throughput.world/blog/ai-in-retail-supply-chain/>.

de afaceri pentru a transforma datele istorice în informații relevante, cu rol de suport decizional într-un mediu economic dinamic și competitiv.⁷

2.2.2. Tipuri de abordări în Machine Learning utilizate în prognoza cererii

Machine Learning poate fi organizat în mai multe paradigme care diferă în funcție de natura datelor și de modul în care acestea sunt utilizate pentru învățare. (Fig.2.)

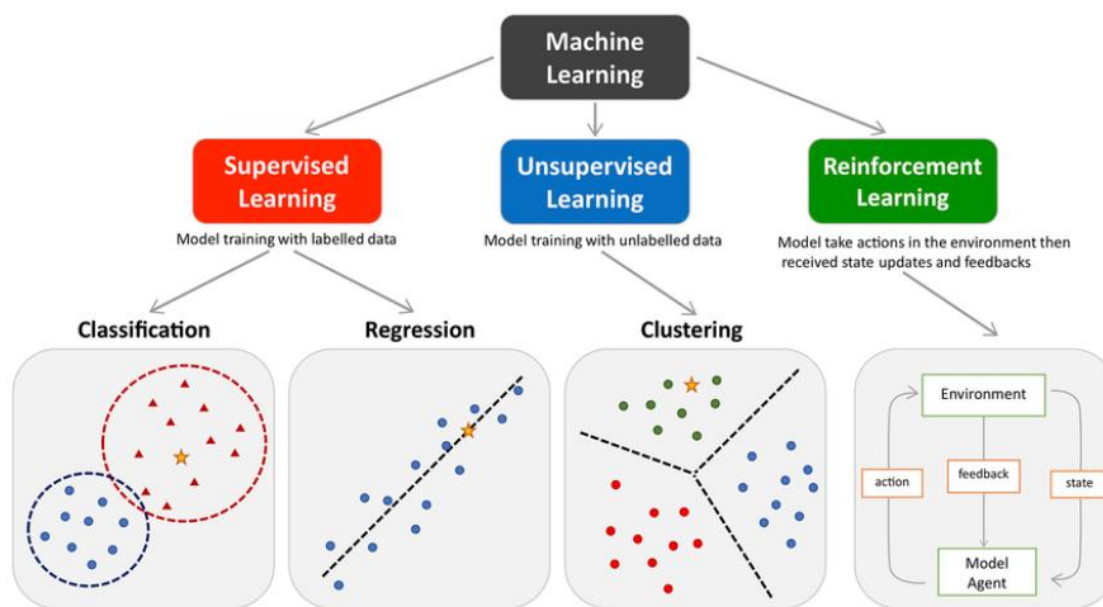


Fig. 2 Principalele tipuri de machine learning

Învățarea supervizată (supervised learning) este cea mai utilizată abordare în prognoza seriilor temporale atunci când scopul este să se prezică valori viitoare pe baza unor date istorice etichetate. În acest caz, datele de intrare și ieșire sunt cunoscute, iar modelul învață relația dintre variabile folosind tehnici de regresie sau alte algoritmi de predicție. Această paradigmă este esențială pentru modele care transformă datele secvențiale în exemple de predicție, cum ar fi regresia sau rețelele neuronale folosite pentru a estima cererea viitoare pe baza observațiilor trecute.⁸

In contextul algoritmilor exista variabile dependente / target / date etichetate si separarea setului de date in subseturi de antrenare si testare pentru evaluarea performantei modelului. Exemple de algoritmi : Regresie liniara, Regresie Logistica, k-NN (k-Nearest

⁷ <https://www.ibm.com/think/topics/machine-learning>

⁸ <https://www.geeksforgeeks.org/machine-learning/time-series-forecasting-as-supervised-learning/>

Neighbors), SVM (Support Vector Machines), Decision Tree, Gradient Boosting (XGBoost, LightGBM, CatBoost).

Pe de altă parte, **învățarea nesupervizată (unsupervised learning)** implică modele care explorează structura internă a datelor fără a avea o etichetă ca rezultat. În contextul gestiunii lanțului de aprovizionare, acest tip de abordare poate fi folosit pentru **identificarea de tipare ascunse, gruparea produselor** cu comportamente de vânzare similare sau **detectarea de anomalii** în datele de vânzări sau de inventar. Algoritmi precum K-means sau analiza principală de componente (PCA) pot ajuta la descoperirea acestor tipare fără a avea o ieșire clar definită de predicție.⁹

În cazul algoritmilor de învățare nesupervizată, nu există o variabilă dependentă sau date etichetate, iar analiza se realizează asupra întregului set de date cu scopul de a identifica structuri, tipare sau grupări interne, evaluarea rezultatelor bazându-se pe metrici interne și nu pe separarea clasică în seturi de antrenare și testare.

Încă o paradigmă este **învățarea prin recompensă (reinforcement learning)**, care, spre deosebire de primele două, nu se concentrează pe predicția directă a valorilor viitoare, ci pe învățarea unei politici optime de acțiune într-un mediu dinamic. Deși nu este folosită în mod obișnuit pentru prognoza pură a cererii, cercetările au explorat aplicarea sa în decizii operaționale cum ar fi planificarea comenzilor într-un lanț de aprovizionare sau controlul inventarului în scenarii dinamice. Astfel de abordări presupun interacțiunea cu un mediu ce oferă recompense sau penalizări pe baza acțiunilor întreprinse, ceea ce poate fi relevant pentru optimizarea deciziilor pe termen lung în gestionarea stocurilor.¹⁰

Reinforcement learning este un cadru de învățare în care un agent învață prin interacțiune cu mediul să ia decizii secvențiale, maximizând o recompensă cumulativă pe termen lung. Exemple de algoritmi RL: Q-Learning, Deep Q-Network, Actor–Critic Proximal Policy Optimization .

2.2.3. Serii temporale în retail

Seriile temporale sunt definite ca observații ordonate cronologic, utilizate pentru analizarea și prognozarea evoluției unei variabile în timp. În analiza cererii, seriile temporale sunt

⁹ [Unsupervised Learning in Supply Chain Demand - growth-onomics](#)

¹⁰ <https://doi.org/10.1016/j.tre.2020.101926>

decompozabile în componente distincte, dintre care cercetarile de specialitate identifică explicit trendul, sezonalitatea și componenta aleatorie.¹¹

Sezonalitatea este descrisă ca o variație sistematică și repetitivă care apare la intervale regulate într-o serie temporală. Această componentă este asociată cu efecte calendaristice recurente, cum ar fi luni, trimestre sau zile ale săptămânii, și este prezentă atunci când aceleași tipare se repetă în aceeași perioadă a fiecărui ciclu. Definiția și caracteristicile sezonalității sunt prezentate explicit în Hyndman și Athanasopoulos (2021, Secțiunea 2.3 – “Times series patterns”).

Trendul reprezintă direcția generală de evoluție a unei serii temporale pe termen lung, manifestată printr-o creștere sau o scădere persistentă a nivelului mediu al observațiilor. Trendul reflectă schimbări structurale lente ale procesului generator de date și este distinct de fluctuațiile sezoniere sau neregulate.¹²

În contextul cererii din retail, seriile temporale pot prezenta un comportament particular cunoscut sub denumirea de cerere intermitentă. *Cererea intermitentă* este caracterizată prin apariția frecventă a valorilor zero, urmate de observații nenule care apar la intervale neregulate.

Syntetos și Boylan (2005) descriu cererea intermitentă ca având două dimensiuni distincte: incertitudinea momentului apariției cererii și variabilitatea cantității solicitate atunci când cererea se manifestă. Aceste caracteristici fac ca seriile intermitente să fie dificil de modelat folosind metode standard de prognoză pentru serii temporale continue.

Literatura indică faptul că prezența unui număr mare de observații nule limitează capacitatea modelelor clasice de a identifica corect structuri precum trendul sau sezonalitatea la nivelul seriei individuale.¹³

2.2.4. Metode statistice vs Machine Learning

În literatura de specialitate orientată spre practică, *statistical learning* și *machine learning* sunt prezentate ca domenii apropiate, dar cu obiective și accente diferite. În timp ce *statistical learning* pune accent pe explicarea și interpretarea relațiilor dintre variabilele

¹¹ Hyndman & Athanasopoulos, 2021, Capitolul 3 – “Time series decomposition”

<https://otexts.com/fpp3/decomposition.html>

¹² <https://otexts.com/fpp3/tspatterns.html>

¹³ <https://www.sciencedirect.com/science/article/abs/pii/S0169207004000792?via%3Dihub>

dintr-un set de date, *machine learning* este descris ca fiind centrat pe predicție, clasificare și pe capacitatea algoritmilor de a „învăța” din date.

Statistical learning (analiză statistică). Statistical learning este definit ca o ramură din data science care urmărește să înțeleagă modul în care variabilele (sau seturile de date) se relaționează în interiorul unui set de date și oferă bază teoretică pentru multe tehnici predictive și analitice, inclusiv pentru unele metode utilizate în machine learning. În această abordare, analiza pornește frecvent de la o ipoteză privind structura datelor, iar apoi sunt construite modele statistice pentru a descrie tipare și pentru a realiza predicții în acord cu aceste tipare.

Analiza statistică este utilizată pentru testarea ipotezelor, explorarea datelor atunci când rezultatele nu sunt încă evidente, modelarea relațiilor dintre variabile (ex. regresie, corelații, ANOVA) și identificarea de tendințe și tipare care pot susține predicții viitoare. În privința avantajelor, sunt menționate scalabilitatea la seturi de date de dimensiuni diferite, versatilitatea pe tipuri variate de date (continue, categoriale, serii temporale) și aplicabilitatea metodelor în contexte diferite. Ca limitări, sunt evidențiate dependența de date istorice (și presupunerea că tiparele se mențin) și existența unor ipoteze statistice stricte (de ex., independența variabilelor, distribuții normale), a căror încălcare poate complica analiza și poate reduce fidelitatea reprezentării datelor.

Machine learning. Machine learning este descris ca subdomeniu al inteligenței artificiale care urmărește dezvoltarea de algoritmi capabili să învețe din date și să-și îmbunătățească performanța în timp, fără instrucțiuni explicite pentru fiecare pas. În comparație cu statistical learning, accentul este pus pe identificarea de tipare (inclusiv unele dificil de observat direct) în seturi mari și complexe și pe obținerea de predicții pentru rezultate sau clasificări.

Modelele ML pot lucra cu volume mari de date, inclusiv date ne-structurate (ex. text, imagini), precum și posibilitatea de a se adapta la informație nouă și la aplicații diferite. Ca dezavantaje, este evidențiată dificultatea de a explica transparent procesul intern al unor algoritmi (ceea ce poate îngreuna identificarea erorilor sau justificarea deciziilor) și dependența performanței de calitatea datelor de antrenare; seturile mici sau dezechilibrate pot introduce bias și pot limita generalizarea.

Criterii de alegere între cele două abordări. Sursa recomandă evaluarea: (1) **complexității datelor** (datele slab structurate sau cu dimensionalitate ridicată pot favoriza ML), (2) **nevoii de explicabilitate** (modelele statistice pot oferi mai multă transparență a pașilor de decizie) și (3) **resurselor disponibile** (modelele ML pot solicita resurse computaționale și expertiză,

în timp ce modelele statistice pot necesita mai puține resurse, dar presupun respectarea anumitor ipoteze).¹⁴

2.2.5. Modele utilizate în cercetarile existente

ARIMA (AutoRegressive Integrated Moving Average)

Modelul ARIMA este un model statistic clasic utilizat pentru analiza și prognoza seriilor temporale univariate, fiind fundamentat pe metodologia Box–Jenkins. Denumirea **ARIMA** provine din combinarea termenilor *AutoRegressive* (AR), *Integrated* (I) și *Moving Average* (MA) și reflectă structura matematică a modelului. Modelul este definit prin parametrii (**p**, **d**, **q**), care controlează modul în care valorile trecute ale seriei și erorile asociate sunt utilizate în estimarea valorilor viitoare.¹⁵

Componenta autoregresivă, de ordin **p**, descrie relația liniară dintre valoarea curentă a seriei și un număr finit de observații anterioare. Această componentă permite captarea dependenței temporale directe dintre observațiile succesive. Parametrul **d** reprezintă gradul de diferențiere aplicat seriei pentru a elimina non-staționaritatea, fiind esențial pentru stabilizarea mediei și a varianței în timp. Componenta de medie mobilă, de ordin **q**, modelează influența termenilor de eroare din perioadele anterioare asupra valorii curente.

¹⁶

Modelul ARIMA este aplicabil în situațiile în care seria temporală nu prezintă sezonaltate sau în care efectele sezoniere au fost eliminate prin preprocesare. În documentațiile tehnice, se subliniază faptul că performanța modelului depinde de identificarea corectă a ordinilor **p**, **d** și **q**, pe baza analizei autocorelației și autocorelației parțiale.

Modelul ARIMA realizează predicțiile pe baza valorilor anterioare ale seriei temporale și a erorilor de estimare din trecut. Într-o manieră intuitivă, acesta poate fi privit ca un mecanism care combină informația recentă privind evoluția cererii cu corecții bazate pe abaterile anterioare ale modelului. Spre deosebire de modelele bazate pe rețele neuronale,

¹⁴ [Statistical Learning vs. Machine Learning: What's the Difference? | Coursera](#)

¹⁵ <https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html>

¹⁶ <https://otexts.com/fpp2/arima.html>

ARIMA nu învață tipare complexe, ci se concentrează pe relațiile locale dintre observațiile succesive, ceea ce îl face adecvat pentru serii relativ stabile, fără trend pronunțat.

Pot apărea limitări în situațiile în care datele sunt puternic sezoniere sau au un comportament neliniar, cazuri în care acuratețea prognozelor poate scădea.

SARIMA (Seasonal AutoRegressive Integrated Moving Average)

Pentru seriile temporale care prezintă un comportament sezonier regulat, modelul ARIMA este extins la forma **SARIMA** (*Seasonal AutoRegressive Integrated Moving Average*). Acest model integrează explicit sezonalitatea prin introducerea unui set suplimentar de parametri sezonieri și este notat sub forma **ARIMA(p, d, q) × (P, D, Q, s)**.

Parametrii P, D, Q reprezintă analogii sezonieri ai componentelor nesezoniere:

- P indică ordinul componentei autoregresive sezoniere (*Seasonal AR*), care modelează relația dintre observații separate printr-un ciclu sezonier complet,
- D reprezintă gradul de diferențiere sezonieră (*Seasonal Integrated*), utilizat pentru eliminarea tendințelor sezoniere persistente,
- Q indică ordinul componentei de medie mobilă sezoniere (*Seasonal MA*), asociată termenilor de eroare la nivel sezonier.

Parametrul s (denumit frecvent și **m**) reprezintă perioada sezonality și corespunde numărului de observații care formează un ciclu complet. De exemplu, pentru date lunare cu sezonality anuală, valoarea lui s este 12.¹⁷

În această notare:

- SAR provine din *Seasonal AutoRegressive*,
- I și D se referă la operațiile de integrare (diferențiere),
- MA desemnează componenta de medie mobilă,
- M este utilizat pentru a indica frecvența sezonieră a seriei.

Modelul SARIMA permite modelarea simultană a dependențelor pe termen scurt și a structurii sezoniere recurente, fiind utilizat frecvent în aplicații de prognoză economică și operațională.

¹⁷ <https://otexts.com/fpp2/seasonal-arima.html>

Exponential Smoothing (Holt–Winters)

Metodele de *exponential smoothing* constituie o clasă distinctă de modele statistice pentru prognoza seriilor temporale, bazate pe ideea de ponderare exponențială a observațiilor trecute. Metoda Holt–Winters extinde netezirea exponențială simplă prin includerea explicită a unei componente de trend și a unei componente sezoniere.¹⁸

În această abordare, valorile recente ale seriei primesc o pondere mai mare decât observațiile mai vechi, iar nivelul, trendul și sezonaliitatea sunt actualizate recursiv la fiecare pas temporal. Documentația statsmodels precizează că metoda poate fi utilizată cu sezonaliitate aditivă sau multiplicativă, în funcție de modul în care amplitudinea fluctuațiilor sezoniere variază în timp.

Holt–Winters este frecvent utilizată ca metodă de referință în studiile comparative de prognoză, datorită simplității sale și a capacității de a modela direct structuri sezoniere regulate.

Modele de Machine Learning pentru prognoza seriilor temporale

XGBoost (Extreme Gradient Boosting)

XGBoost este un algoritm de *gradient boosting* bazat pe arbori de decizie, conceput pentru a optimiza performanța și eficiența în probleme de regresie și clasificare. Documentația oficială descrie XGBoost ca un cadru de învățare care combină mai mulți arbori slabi într-un model puternic, prin minimizarea iterativă a unei funcții obiectiv .

În prognoza seriilor temporale, XGBoost este utilizat prin reformularea problemei într-un cadru de învățare supravegheată. Aceasta presupune construirea unui set de variabile explicative derivate din istoricul seriei, precum valori întârziate (*lag features*) și caracteristici calendaristice. Modelul nu presupune staționaritatea seriei și poate surprinde relații neliniare între variabilele explicative și variabila țintă.¹⁹

În implementarea XGBoost pentru prognoza seriilor temporale, un pas central este construirea explicită a unor variabile care să redea dependențele în timp. O tehnică uzuală este generarea **lag-urilor**, unde valorile trecute ale variabilei țintă sunt introduse ca predictor (de exemplu lag_1, lag_2, ..., până la un număr de pași definit). Practic, pentru

¹⁸ <https://www.statsmodels.org/stable/generated/statsmodels.tsa.holtwinters.ExponentialSmoothing.html>

¹⁹ <https://xgboost.readthedocs.io/en/stable/>

fiecare moment de timp se atașează una sau mai multe valori din trecut, astfel încât modelul să poată învăța relații dintre nivelul curent și istoricul recent, inclusiv tendințe istorice și dependențe temporale.

În completare, se utilizează frecvent **statistici de tip rolling**, în special **rolling mean (moving average)**, care creează o caracteristică nouă prin calcularea mediei țintei pe o fereastră glisantă de dimensiune fixă (de exemplu, 3, 5 sau 7 observații). Această medie mobilă are rolul de a **netezi** seria, reducând fluctuațiile pe termen scurt și evidențiind mai clar direcția generală a evoluției (trenduri și tipare), oferind astfel modelului un semnal mai stabil decât valorile punctuale.

În cadrul antrenării, performanța poate fi optimizată prin reglarea hiperparametrilor (de exemplu, learning rate, adâncimea maximă a arborilor și parametri de regularizare), folosind căutare de tip grid search sau random search.

În evaluarea prognozelor sunt menționate metrice uzuale precum MAE, RMSE și MAPE, care cuantifică diferențele dintre valorile reale și predicții. De asemenea, este subliniată utilitatea scorurilor de importanță a variabilelor furnizate de model pentru selectarea predictorilor relevanți, precum și rolul regularizării (L1/L2) în reducerea supraînvățării. Ca limitare, este menționat faptul că XGBoost poate avea dificultăți în captarea dependențelor pe termen lung, situație în care pot fi considerate modele secvențiale precum RNN/LSTM.²⁰

Random Forest Regression

Random Forest este un algoritm de tip *ensemble learning* care utilizează un ansamblu de arbori de decizie antrenați pe eșantioane bootstrap ale datelor. Predicția finală este obținută prin agregarea (de regulă media) predicțiilor individuale ale arborilor. Această abordare reduce varianța modelului și îmbunătățește stabilitatea rezultatelor.

În aplicațiile de prognoză a seriilor temporale, Random Forest este utilizat într-un mod similar cu XGBoost, prin transformarea seriei într-un set de date supravegheat. Documentația subliniază faptul că modelul nu impune ipoteze stricte privind distribuția datelor sau structura temporală, fiind astfel potrivit pentru serii cu comportament neliniar.

²¹

²⁰ <https://www.analyticsvidhya.com/blog/2024/01/xgboost-for-time-series-forecasting/>

²¹ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

LSTM (Long Short-Term Memory)

Modelul **Long Short-Term Memory (LSTM)** este o extensie a rețelelor neuronale recurente (RNN), concepută pentru a depăși limitările acestora în modelarea seriilor temporale. Rețelele RNN clasice întâmpină dificultăți în reținerea informației relevante pe intervale lungi de timp, fenomen cunoscut sub denumirea de *vanishing gradient*. LSTM rezolvă această problemă prin introducerea unui mecanism explicit de memorie.

LSTM este o arhitectură de rețele neuronale recurente concepută pentru modelarea datelor secvențiale și a dependențelor pe termen lung. Documentația TensorFlow descrie LSTM ca o extensie a RNN-urilor clasice, care utilizează mecanisme de tip *input gate*, *forget gate* și *output gate* pentru a controla fluxul informației în timp.

În prognoza seriilor temporale, LSTM este utilizat pentru a învăța relații temporale complexe direct din date, fără a necesita specificarea explicită a componentelor de trend sau sezonaliitate. Această capacitate face ca modelele LSTM să fie frecvent investigate în studii recente de prognoză bazată pe inteligență artificială.

Principiul de funcționare

Arhitectura LSTM este construită în jurul unei **celule de memorie**, care permite stocarea informației pe termen lung. Actualizarea acestei memorii este controlată de trei mecanisme denumite **porți (gates)**, care reglează fluxul de informație în rețea:

- **Forget gate** – decide ce informații din starea anterioară sunt eliminate;
- **Input gate** – stabilește ce informații noi sunt adăugate în memoria celulei;
- **Output gate** – determină ce parte din informația memorată este utilizată pentru generarea ieșirii.

Prin acest mecanism, LSTM poate păstra informația relevantă și elimina zgomotul, adaptându-se dinamic la evoluția seriei temporale.²²

Prophet

Prophet este o metodă de prognoză a seriilor temporale construită pe un **model aditiv decompozabil**, în care observațiile sunt explicate ca sumă de componente (trend + sezonaliități + efecte de sărbători/evenimente), cu parametri interpretați ușor și ajustabili. În implementarea Python, Prophet folosește o interfață de tip *scikit-learn*: se creează un obiect Prophet, apoi se aplică `fit()` pentru antrenare și `predict()` pentru generarea prognozei. Intrarea este un DataFrame cu două coloane obligatorii: `ds` (datestamp, într-un

²² Brownlee, J. (2017). *An Introduction to Long Short-Term Memory Networks (LSTM)*. Towards Data Science. <https://towardsdatascience.com/an-introduction-to-long-short-term-memory-networks-lstm-27af36dde85d/>

format compatibil cu pandas, de exemplu YYYY-MM-DD sau YYYY-MM-DD HH:MM:SS) și y (valoarea numerică de prognozat).

Modelul include explicit un **trend** care poate fi configurat și ajustat în timp (în lucrarea de prezentare este descris ca parte dintr-un model decompozabil, cu parametri interpretabili, gândit pentru utilizare la scară).²³

Pentru **sezonalitate**, documentația arată că Prophet ajustează implicit sezonabilitate **săptămânală** și **anuală** dacă există suficient istoric (peste două cicluri), și poate ajusta și sezonabilitate **zilnică** pentru serii sub-zilnice; în plus, pot fi adăugate sezonaliități personalizate (ex. lunară/trimestrială/orară) prin `add_seasonality`.

Prophet permite și includerea **efectelor de sărbători/evenimente** (holiday effects) ca termeni separați ai modelului, astfel încât impactul acestor zile poate fi estimat explicit în prognoză.²⁴

În învățarea automată, evaluarea performanței unui model este esențială, deoarece arată cât de bine reușește acesta să prezică sau să clasifice datele. Dintre numeroșii indicatori disponibili, MAE (Mean Absolute Error), MSE (Mean Squared Error) și RMSE (Root Mean Squared Error) sunt printre cei mai utilizați.

MAE (Mean Absolute Error) măsoară **mărima medie a erorilor** dintre valori reale și predicții, fără să țină cont de semn (nu contează dacă modelul a supraestimat sau subestimat). Se calculează ca **media diferențelor absolute** dintre y și $\hat{y}$, ceea ce îl face ușor de interpretat în practică: dacă MAE este 5, înseamnă că, în medie, predicțiile sunt la 5 unități distanță de valorile reale. MAE este apreciat pentru **interpretabilitate** și pentru faptul că este **mai robust la outlieri** decât MSE/RMSE, deoarece nu pătrățește erorile. Este recomandat când se dorește o înțelegere directă a erorii medii și când costul erorilor crește aproximativ liniar (adică o eroare dublă „costă” aproximativ dublu).

MSE (Mean Squared Error) calculează **media erorilor pătrate** dintre valori reale și predicții. Pătrarea erorilor produce un efect de **amplificare**: erorile mari devin mult mai importante decât erorile mici, ceea ce face MSE potrivit în situații în care abaterile mari sunt deosebit de nedorite. Un alt motiv pentru utilizarea MSE este natura sa matematică: fiind **derivabil**, este adesea folosit ca funcție de pierdere în optimizare (de exemplu, în algoritmi de tip gradient descent), deoarece derivata este relativ ușor de calculat. În practică, MSE este util atunci când se urmărește penalizarea puternică a deviațiilor mari și când se dorește un criteriu convenabil pentru antrenare/optimizare.

²³ https://facebook.github.io/prophet/docs/quick_start.html#python-api

²⁴

https://facebook.github.io/prophet/docs/seasonality%2C_holiday_effects%2C_and_regressors.html?utm_source=chatgpt.com

RMSE (Root Mean Squared Error) este rădăcina pătrată a **MSE**. Avantajul principal este că aduce eroarea „înapoi” în aceeași scară/unitate ca datele originale, fiind astfel mai ușor de interpretat decât MSE (care este în unități pătrate). RMSE păstrează sensibilitatea la erori mari (la fel ca MSE), dar oferă o măsură mai intuitivă a mărimii erorii. Este preferat când vrei o **penalizare mai puternică pentru erori mari**, dar cu un rezultat exprimat în unitățile țintei, astfel încât să poată fi interpretat direct (de ex. „în medie, abaterea tipică este ~X unități, cu accent pe erorile mari”).²⁵

2.3. Aplicații practice și rezultate raportate în cercetarile existente

Potrivit unei analize efectuate în 178 de companii de retail din Europa și America de Nord, sistemele hibride PLS-SEM (Partial Least Squares Structural Equation Modeling - o metodă statistică pentru analiza relațiilor complexe dintre variabile) și sistemele de prognoză bazate pe machine learning au contribuit la o creștere de 3,7% a marjelor brute medii, în același timp reducând pierderile financiare legate de reducerile de preț cu aproximativ 28,6 milioane EUR anual pentru marii comercianți din retail. Aceste câștiguri de performanță provin în principal din capacitatea sporită de a detecta relații neliniare dintre variabilele pieței și modelele de comportament ale consumatorilor, pe care metodele tradiționale nu reușesc în mod constant să le identifice.²⁶

În articolul din Procurement Magazine, Schwarz IT (organizația digitală din spatele unor retaileri precum Lidl și Kaufland) este prezentată ca folosind planificare a cererii bazată pe AI pentru a îmbunătăți managementul cererii și al re aprovizionării, cu impact asupra procurement, supply chain și inventory management. Cazul este relevant și pentru context european), deoarece Kaufland operează în mai multe țări, printre care este menționată explicit și România.

Elementele-cheie ale soluției descrise sunt:

- Obiectivul implementării: introducerea unei soluții AI pentru a anticipa mai bine cererea și a susține decizii mai bune de aprovizionare și re aprovizionare, în condiții de schimbare rapidă a comportamentului clienților și complexitate ridicată a lanțului

²⁵ https://dev.to/mondal_sabbha/understanding-mae-mse-and-rmse-key-metrics-in-machine-learning-4la2

²⁶ <https://www.ijssat.org/papers/2025/1/2644.pdf>

de aprovizionare (inclusiv produse perisabile) și la scară mare (peste 1.500 de magazine, în mai multe țări).

- Componenta tehnologică menționată: articolul descrie utilizarea modulului SAP Unified Demand Forecast (UDF), indicând că acesta folosește algoritmi statistici și tehnici de machine learning pentru a analiza date istorice și alți factori relevanți și pentru a genera prognoze care sprijină procurement, supply chain și inventarul.
- Scara de procesare și orizontul de prognoză: este raportat că soluția calculează până la 35 milioane combinații magazin–produs pe zi, modelează un istoric de 800 de zile și estimează vânzările așteptate pentru 100 de zile înainte.
- Legătura prognoză → decizie de stoc: articolul precizează că ordinele de achiziție către furnizori și aprovizionarea magazinelor sunt controlate pe baza prognozelor „pentru viitorul apropiat” sau chiar pentru zilele următoare, atunci când este necesar.

Din perspectiva rezultatelor, materialul afirmă că abordarea contribuie la îmbunătățirea managementului stocurilor, inclusiv prin reducerea risipei și prin „achizitii inteligente” adaptat nevoilor magazinelor și clienților; este menționată și îmbunătățirea re aprovizionării și a serviciului către clienți în urma prognozei mai bune a cererii.²⁷

Un exemplu de material publicat în limba română, utilizabil ca referință aplicată pentru contextul lucrării, este articolul Slimstock despre „prognoza cererii” (Phipps, 2025). Acesta definește prognozarea cererii ca proces de **estimare a nevoilor viitoare de produse** pe baza datelor, cu rol în fundamentarea deciziilor privind **stocurile, producția și finanțele**, subliniind că activitatea presupune **analiză continuă** și poate utiliza metode precum **serii temporale, judecată/estimare și machine learning**, iar acuratețea poate fi îmbunătățită prin agregarea datelor și alegerea unor intervale de timp adecvate.

În același articol, prognoza cererii este prezentată ca element central pentru decizii operaționale (de exemplu, cantități comandate, producție, prețuri, marketing) și este asociată cu obiectivul de a evita situații precum **suprastocarea și epuizarea stocurilor**, prin estimări cât mai apropiate de necesarul real.

Articolul enumeră și beneficii tipice ale prognozei cererii, precum **gestionarea mai bună a inventarului** (niveleuri de stoc mai bine controlate), **planificare îmbunătățită a producției**, **aliniere între echipe** (vânzări, marketing, finanțe, operațiuni), precum și suport pentru decizii în lanțul de aprovizionare și pentru **vizibilitatea financiară**.²⁸

²⁷ <https://procurementmag.com/articles/schwarz-it-better-inventory-management-through-ai>

²⁸ <https://www.slimstock.com/ro/blog/prognoza-cererii/>

Capitolul 3. Metodologia de cercetare

3.1. Descriere

Cercetarea realizată în cadrul acestei lucrări este de tip aplicativ, bazată pe analiza datelor provenite dintr-un sistem informatic utilizat în domeniul retail. Abordarea aleasă are ca obiectiv analizarea și evaluarea utilizării metodelor predictive asupra datelor istorice de vânzări, în contextul optimizării proceselor de gestiune a stocurilor într-un sistem informatic integrat.

Studiul de caz este fundamentat pe date reale din cadrul firmei în care lucrez, extrase din baza de date operațională a unui client din retail, ceea ce permite analizarea comportamentului vânzărilor într-un cadru concret și relevant din punct de vedere organizațional. Componenta cantitativă a cercetării este justificată de natura datelor analizate, acestea fiind structurate numeric și organizate temporal, fiind adecvate aplicării metodelor statistice și de machine learning.

Datele utilizate

Setul de date utilizat în cercetare provine dintr-o bază de date relațională aferentă unui singur magazin din domeniul retail. Acesta conține informații privind vânzările produselor pe o perioadă de doi ani, începând cu anul 2024, fiind organizat la nivel de articol și timestamp zilnic. Datele includ indicatori cantitativi și valorici, necesari pentru analiza comportamentului vânzărilor și pentru aplicarea modelelor predictive.

Portofoliul de produse analizat este caracterizat de o diversitate ridicată, incluzând un număr mare de articole aparținând mai multor furnizori. Pentru a reduce complexitatea analizei și pentru a asigura relevanța rezultatelor, a fost aplicată o analiză Pareto, atât la nivel general, cât și la nivelul unui singur brand selectat. Această etapă a permis identificarea unui subset de articole reprezentative, care concentrează o parte semnificativă din volumul vânzărilor.

Principiul Pareto (cunoscut și sub numele de regula 80:20, legea câtorva factori vitali și principiul rarității factorilor) afirmă că, pentru multe rezultate, aproximativ 80% din consecințe provin din 20% din cauze („câteva cauze vitale”).²⁹

²⁹ [Pareto principle - Wikipedia](#)

În contextul setului de date analizat în cadrul acestei lucrări, aplicarea principiului Pareto asupra vânzărilor a evidențiat faptul că un subset restrâns de articole contribuie într-o măsură semnificativă la volumul total al vânzărilor. Mai precis, analiza a arătat că aproximativ 20% dintre articolele comercializate generează o proporție dominantă din vânzările totale ale magazinului, confirmând relevanța acestui principiu în structura cererii specifice domeniului retail. Această observație a stat la baza selecției articolelor utilizate în etapele ulterioare de analiză și modelare predictivă.

Etapele cercetării

Procedura de cercetare a fost structurată în mai multe etape succesive, menite să asigure coerența procesului de analiză și reproducibilitatea rezultatelor.

În prima etapă, a fost realizată extragerea datelor din baza de date operațională, utilizând interogări SQL. Acestea au fost concepute pentru a agrega informațiile la nivel de articol și perioadă de timp, precum și pentru a identifica articolele relevante din perspectiva contribuției la vânzări.

În etapa următoare, a fost aplicată analiza Pareto, cu scopul de a selecta articolele care generează o pondere semnificativă din vânzările totale. Analiza a fost realizată inițial asupra întregului set de produse, iar ulterior restrânsă la nivelul unui singur brand, pentru a obține un set de date mai omogen și adecvat modelării predictive.

Ulterior, datele au fost prelucrate și organizate într-un format adecvat analizei, fiind eliminate înregistrările irelevante și asigurată consistența temporală a seriilor analizate. Setul de date rezultat a constituit baza pentru etapele de analiză exploratorie și de modelare predictivă.

Analiza datelor a fost realizată utilizând atât metode statistice, cât și metode de machine learning, aplicate asupra seriilor temporale de vânzări. În etapa de analiză exploratorie au fost utilizate tehnici descriptive pentru identificarea distribuției vânzărilor, a sezonality și a variațiilor în timp.

Pentru prognoza cererii, au fost aplicate modele predictive asupra seriilor temporale aferente articolelor selectate prin analiza Pareto. Modelele au fost antrenate pe baza datelor istorice și evaluate în funcție de capacitatea acestora de a surprinde evoluția vânzărilor în timp. Această abordare permite compararea performanței metodelor tradiționale cu cea a modelelor bazate pe machine learning, în contextul datelor reale dintr-un sistem ERP.

3.2. Instrumente și tehnologii utilizate

Prelucrarea și analiza datelor au fost realizate utilizând limbajul de programare **Python**, datorită flexibilității acestuia și a suportului extins pentru analiza seriilor temporale și dezvoltarea modelelor predictive. Interogarea bazei de date a fost realizată prin intermediul limbajului **SQL**.

O parte dintre bibliotecile **Python** folosite :

- **pandas** – utilizată pentru manipularea datelor și operații specifice seriilor temporale, inclusiv re-eșantionare la frecvență zilnică (resample), deplasare în timp pentru lag-uri (shift) și agregări pe ferestre mobile (rolling).
- **NumPy** – utilizată ca bibliotecă de bază pentru calcul numeric și operații eficiente pe vectori/array-uri, necesare în transformări, calcule de metrici și prelucrări auxiliare.
- **scikit-learn** – utilizată pentru metrici de evaluare ale regresiei (de exemplu MAE, MSE/RMSE, R^2) și pentru proceduri de validare care respectă ordinea temporală (de exemplu TimeSeriesSplit, util pentru a evita antrenarea pe date din viitor).³⁰
- **XGBoost (XGBRegressor)** – utilizat pentru prognoză în regim de învățare supravegheată, prin interfața compatibilă cu scikit-learn; parametrii de antrenare și regularizare sunt controlați explicit din estimator.
- **statsmodels** – utilizat pentru diagnosticarea proprietăților seriei, inclusiv testul Augmented Dickey–Fuller (ADF) pentru verificarea existenței unei rădăcini unitare.
- **TensorFlow / Keras (LSTM)** – utilizat pentru implementarea și testarea unui model neuronal de tip LSTM, adecvat prelucrării secvențelor și dependențelor temporale.
- **Matplotlib** – utilizată pentru generarea graficelor necesare interpretării seriei și a rezultatelor (de exemplu linii, comparații real vs. predicție), prin interfața pyplot și funcții precum plot.
- **Streamlit** este o bibliotecă Python open-source pentru construirea de aplicații web interactive folosind doar Python. Este ideală pentru crearea de tablouri de bord, aplicații web bazate pe date, instrumente de raportare și interfețe utilizator interactive fără a fi nevoie de HTML, CSS sau JavaScript.³¹

Medii de dezvoltare și testare

- **Visual Studio Code** – utilizat ca mediu principal de dezvoltare, împreună cu extensiile dedicate Python și Jupyter, care oferă suport pentru editare, rulare, depanare și gestionarea mediilor Python.

³⁰ <https://www.geeksforgeeks.org/python/libraries-in-python/>

³¹ <https://www.geeksforgeeks.org/python/a-beginners-guide-to-streamlit/>

- **Jupyter Notebook** – utilizat pentru antrenare și experimentare iterativă; formatul de notebook permite combinarea codului executabil cu text narativ și output-uri (tabele, grafice), facilitând documentarea pașilor de analiză.

3.3. Variabile și indicatori analizați

Variabila țintă

Variabila țintă a cercetării este reprezentată de cantitatea vândută a fiecărui articol, înregistrată la nivel temporal (zilnic). Această variabilă reflectă direct nivelul cererii și constituie principalul indicator utilizat pentru prognoza vânzărilor.

Alegerea cantității vândute ca variabilă țintă este justificată de rolul său central în procesele de gestiune a stocurilor, întrucât aceasta influențează deciziile privind reprovizionarea, dimensionarea stocurilor și prevenirea situațiilor de ruptură de stoc sau suprastocare. Variabila este utilizată sub formă de serie temporală, permițând analiza evoluției cererii în timp și aplicarea metodelor predictive.

Predictori

Setul de predictori utilizat în cadrul modelelor predictive este format din variabile derivate din structura temporală a datelor și din informațiile disponibile în baza de date.

Principalii predictori utilizați includ:

- Dimensiunea temporală, reprezentată prin timestamp-ul asociat fiecărei înregistrări, utilizată pentru ordonarea și structurarea seriilor temporale;
- Indicatori temporali derivați, precum luna sau perioada calendaristică, utilizați pentru surprinderea variațiilor sezoniere ale vânzărilor;
- Valori istorice ale variabilei țintă, utilizate implicit de modelele de prognoză a seriilor temporale pentru identificarea tiparelor recurente și a dependențelor în timp.

Predictorii selectați sunt adecvați scopului cercetării, întrucât permit modelarea evoluției cererii pe baza comportamentului istoric al vânzărilor, fără a introduce complexitate suplimentară prin variabile care nu sunt disponibile în mod curent în sistemele informatice de tip ERP.

Capitolul 4. Rezultate și aplicația dezvoltată

4.1 Generarea setului de date si pre-processing

Primul pas in dezvoltarea aplicatiei este reprezentat de alegerea datelor, interogarea bazelor de date pentru a obtine un set optim ce este exportat sub forma de fisier csv, pentru usurinta utilizarii strict in cadrul proiectului . Acest pas poate fi automatizat, creandu-se script-uri python ce export automat seturile de date sau realizeaza functii de merge intre seturile de date cu informatiile noi adaugate si cele deja existente sau prin conexiunea direct catre baza – proces ce poate fi cosisitor din punctul de vedere al operatiunilor rulate si al timpului de executie.

Având în vedere diversitatea ridicată a articolelor disponibile la nivelul magazinului analizat, utilizarea întregului set de produse ar fi condus la o complexitate ridicată a procesului de analiză și la dificultăți în interpretarea rezultatelor. Selecția articolelor a fost realizată prin aplicarea metodei Pareto, utilizată pentru identificarea produselor cu impact semnificativ asupra volumului total al vânzărilor.

Exemplul din Fig. 3 arata o prima incercare a principiului pareto pentru grupele de articol, ce apoi a fost realizat pentru branduri / furnizori si pentru articolele achizitionat pe un singur furnizor. Această abordare a permis obținerea unui set de date mai omogen și mai adecvat pentru analiza seriilor temporale și pentru implementarea modelelor predictive, in figura 4 este o parte din setul de date rezultat ce contine primele articole, cele mai importante din punctul de vedere al analizei pareto, ce vor fi folosite mai departe in lucrare, in analiza singulara si predictii pe un singur articol.

```

select *
from (
  with baza as (
    select
      a.grupa,
      sum(v.val_iesire_fara_tva) as total_vanzari
    from f_vanzare_est_si@warehouse v
    join d_articol@warehouse a
    on a.id_articol = v.id_articol
    where v.id_gestiune = 2
    group by a.grupa
  ),
  total as (
    select sum(total_vanzari) as total_general
    from baza
  )
  select
    b.grupa,
    b.total_vanzari,
    b.total_vanzari / t.total_general as pondere,
    sum(b.total_vanzari) over (
      order by b.total_vanzari desc
    ) / t.total_general as pondere_cumulata
  from baza b
  cross join total t
  order by b.total_vanzari desc
)
where pondere_cumulata <= 0.8;

```

Fig. 3 Interogarea datelor aplicand Pareto

	A	D	E	F
1	ID_ARTICOL	TOTAL_VANZARI	PONDERE	PONDERE_CUMULATA
2	147807	482439.43	0.020452779	0.020452779
3	181925	469426.71	0.019901112	0.040353891
4	118033	412994.04	0.017508677	0.057862568
5	69344	399572.18	0.016939664	0.074802232
6	190760	392814.25	0.016653165	0.091455396
7	147808	377916.81	0.016021595	0.107476991
8	67303	356541.47	0.015115398	0.122592389
9	94871	287981.83	0.012208846	0.134801236
10	140755	286061.2	0.012127422	0.146928658
11	147790	278235.5	0.011795656	0.158724313
12	94869	275438.94	0.011677097	0.17040141
13	144602	272906.46	0.011569734	0.181971144
14	67074	260218.2285	0.011031822	0.193002966
15	67277	259333.06	0.010994296	0.203997262

Fig. 4 Set de date rezultat dupa pareto

Dupa alegerea articolelor, urmatorul pas a fost exportarea datelor detaliate cu privire la vanzari pentru toate articolele alese, apoi sparte pe fiecare articol.

Pre-processing

Înainte de aplicarea modelelor de predicție, setul de date a fost supus unui proces de preprocesare, cu scopul de a asigura calitatea și consistența informațiilor utilizate în analiză. Această etapă este esențială în cazul seriilor temporale, deoarece modelele predictive sunt sensibile la discontinuități, erori de structură temporală și inconsecvențe ale valorilor.

În cadrul acestei etape, a fost analizat gradul de completitudine al datelor, prin evaluarea existenței valorilor lipsă la nivelul variabilelor utilizate.

Valorile lipsă au fost tratate diferențiat, în funcție de semnificația economică a variabilelor. Pentru cantitățile vândute, valorile lipsă au fost considerate absențe reale ale cererii și imputate cu zero. Valorile financiare asociate vânzărilor au fost imputate condiționat, doar în situațiile în care cantitatea vândută a fost zero. Valorile lipsă aferente reducerilor comerciale au fost tratate pe baza ratei de discount, pentru a evita introducerea unor distorsiuni artificiale în date.

```

: # reguli de calitate
checks = {}

# 1) vânzare > 0 dar valoare = 0 (suspect)
checks["qty_pos_val_zero"] = df[(df["CANTITATE"] > 0) & (df["VAL_IESIRE_CU_TVA"] == 0)]

# 2) valoare > 0 dar cantitate = 0 (suspect)
checks["val_pos_qty_zero"] = df[(df["VAL_IESIRE_CU_TVA"] > 0) & (df["CANTITATE"] == 0)]

# 3) stoc final negativ (dacă există în dataset)
if "STOC_FINAL" in df.columns:
    checks["stoc_final_negativ"] = df[df["STOC_FINAL"] < 0]

# 4) discount > 0 dar rata_discount = 0 (suspect)
if "TOTAL_DISCOUNT" in df.columns and "RATA_DISCOUNT" in df.columns:
    checks["disc_pos_rata_zero"] = df[(df["TOTAL_DISCOUNT"] > 0) & (df["RATA_DISCOUNT"] == 0)]

# 5)
checks["stockout_start"] = df[(df["STOC_INITIAL"] == 0) & (df["CANTITATE"] == 0)]
checks["no_sales_but_in_stock"] = df[(df["STOC_INITIAL"] > 0) & (df["CANTITATE"] == 0)]

{k: v.shape[0] for k, v in checks.items()}

: {'qty_pos_val_zero': 0,
  'val_pos_qty_zero': 0,
  'stoc_final_negativ': 0,
  'disc_pos_rata_zero': 0,
  'stockout_start': 0,
  'no sales but in stock': 16}

```

Fig. 5 validarea datelor

De asemenea, datele au fost verificate din punct de vedere al consistenței tipurilor de date și al structurii temporale. Variabila de timp a fost convertită într-un format calendaristic adecvat, permițând ordonarea corectă a observațiilor și utilizarea funcționalităților specifice analizei seriilor temporale.

4.2. Analiza exploratorie a datelor

Analiza exploratorie a datelor a avut ca scop înțelegerea comportamentului vânzărilor în timp și identificarea principalelor caracteristici ale seriei temporale analizate. Această etapă a permis observarea evoluției generale a cantităților vândute, precum și evidențierea variațiilor și fluctuațiilor existente pe parcursul perioadei analizate.

Verificarea anomaliilor a fost realizată folosind grafice de tip scatterplot și boxplot : fig 5 și fig 6. Observațiile atipice identificate în cadrul analizei exploratorii au fost păstrate în setul de date, întrucât acestea reflectă comportamente reale ale cererii și sunt relevante pentru antrenarea modelelor predictive.

Analiza anomaliilor / Outliers (Outlier detection)

```
4]: plt.figure(figsize=(8,4))
plt.scatter(df["DATA"], df["CANTITATE"], alpha=0.6)
plt.title(f"Evoluția cantității - articol balsam")
plt.xlabel("Data")
plt.ylabel("Cantitate")
plt.show()
```

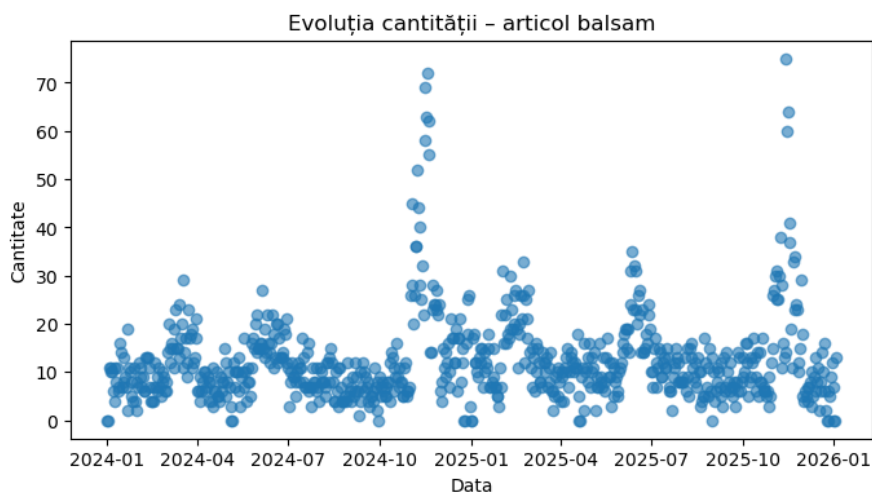


Fig. 6 Analiza anomaliilor

Box plot – CANTITATE (outliers)

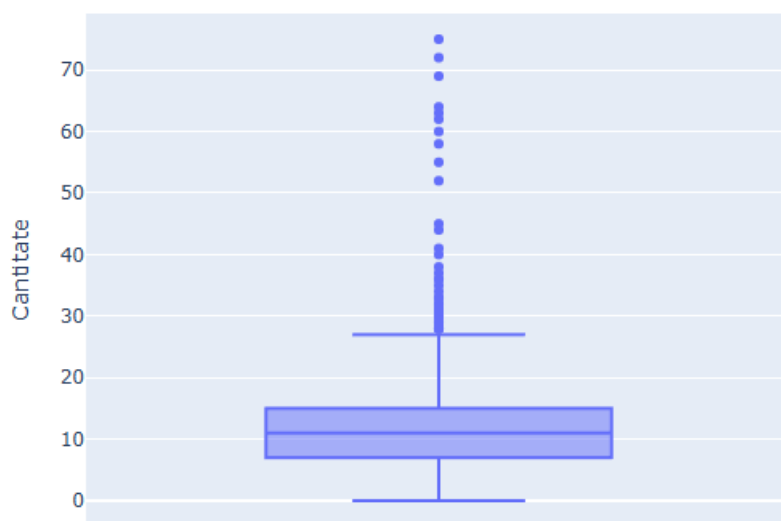


Fig. 7 Boxplot

Prin reprezentări grafice ale seriei temporale, a fost analizată dinamica vânzărilor, fiind evidențiate perioadele cu valori ridicate sau scăzute, precum și stabilitatea relativă a cererii. Această analiză vizuală a oferit un context esențial pentru interpretarea datelor și pentru alegerea abordărilor de modelare adecvate. (fig 7)

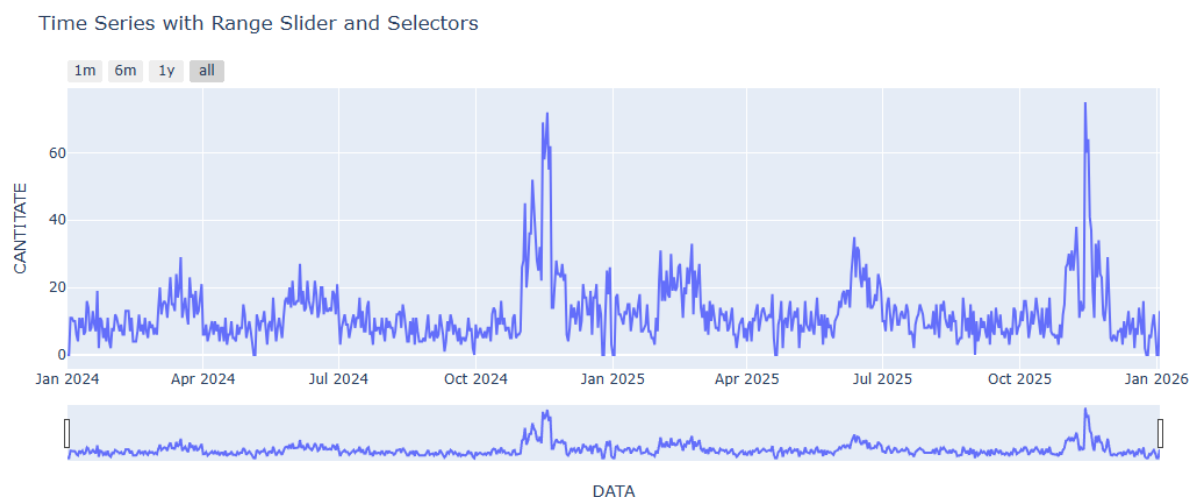


Fig. 8 Grafic Evolutia cantitatii vandute in timp

De asemenea, a fost realizată o analiză a relațiilor dintre variabilele cantitative disponibile, utilizând coeficientul de corelație Pearson. Rezultatele au fost reprezentate sub forma unei hărți de corelație, evidențiind gradul de asociere dintre indicatori. Această analiză a permis identificarea variabilelor puternic corelate și a contribuit la evitarea redundanței informaționale în etapele ulterioare. Pentru evitarea redundanței informaționale, variabilele puternic corelate vor fi filtrate (["STOC_FINAL", "RUPTURA_STOC", "VAL_IESIRE_CU_TVA", "RATA_DISCOUNT"])

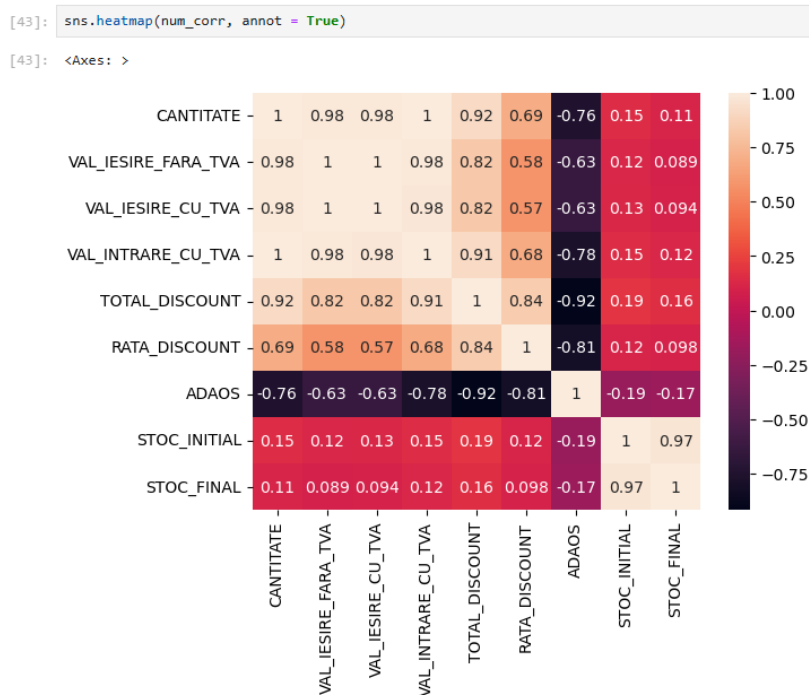


Fig. 9 Corelatie Pearson - Heatmap

Pentru modelele de tip serie temporală, setul de date a fost împărțit în mod cronologic, evitând amestecarea observațiilor. Utilizarea unui random split ar introduce scurgeri de informație temporală și ar conduce la evaluări nerealiste ale performanței modelului.

4.3. Descompunerea seriilor temporale

```
[17]: from statsmodels.tsa.seasonal import STL
```

```
# STL decomposition (daily series)
stl_series = ts.asfreq('D').interpolate('time').fillna(0)
stl = STL(stl_series, period=7, robust=True)
stl_res = stl.fit()

fig = stl_res.plot()
fig.set_size_inches(10, 6)
plt.show()
```

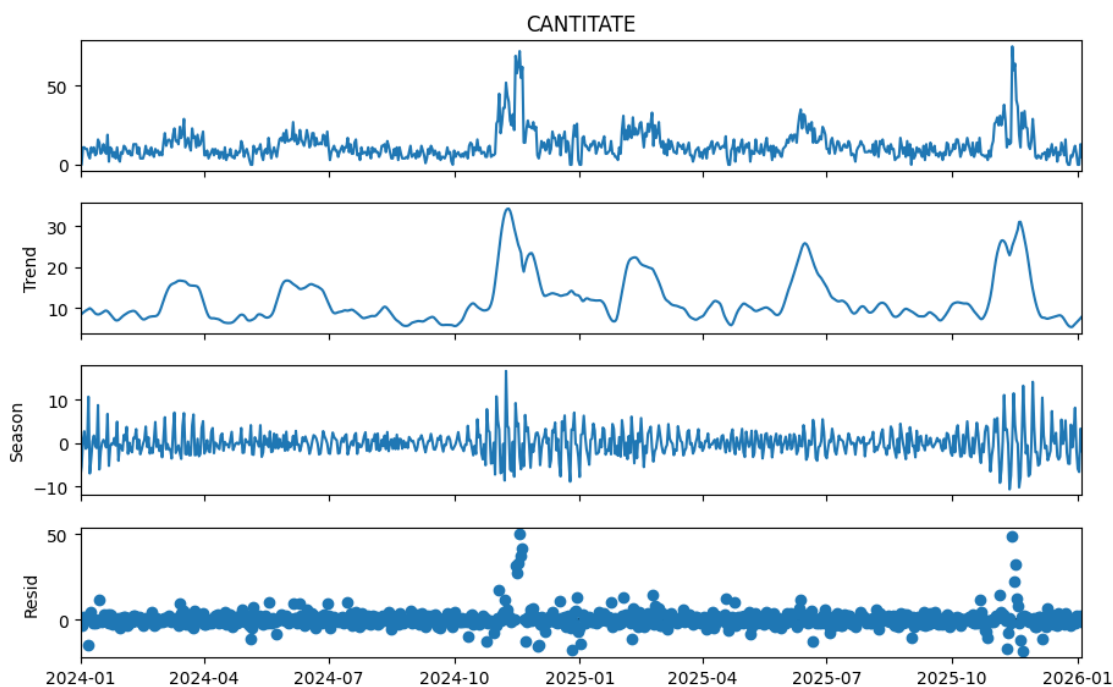


Fig. 10 STL

Decompozitie STL (Observed / Trend / Season / Resid) - Fig 9

Interpretare (period=7) :

- **Observed:** serie cu nivel relativ stabil, dar cu **spike-uri mari** în jurul finalului lui 2024 și finalului lui 2025.
- **Trend:** surprinde schimbări pe termen mediu (urcări/coborâri) și „ignoră” spike-urile, care nu sunt structurale.

- **Season (7 zile):** evidențiază un **ciclu săptămânal**; amplitudinea sezonieră crește în perioadele cu activitate mai intensă (în jurul spike-urilor), ceea ce sugerează că patternul săptămânal există, dar intensitatea lui variază în timp.
- **Resid:** conține majoritatea evenimentelor neexplicate (outlieri / șocuri), mai ales în perioadele de vârf → spike-urile sunt mai degrabă **evenimente punctuale** decât sezonabilitate.

Train Test Split

În modelele de predicție, este o practică obișnuită să se împartă datele în două seturi diferite. Aceste două seturi sunt **setul de antrenament** și **setul de testare**. După cum sugerează și numele, setul de antrenament este utilizat pentru antrenarea modelului, iar setul de testare este utilizat pentru testarea acurateții modelului.

Cel mai comun raport de divizare este **80:20**. Adică 80% din setul de date intră în setul de antrenament, iar 20% din setul de date intră în setul de testare.³²

Pentru a evita parametrul `random_state` din funcția automată `train_test_split`, care implică o amestecare aleatorie a observațiilor și poate provoca pierderea ordinii cronologice în seriile temporale, împărțirea setului de date a fost realizată în mod secvențial, pe baza timpului. Pentru o prognoza de timp concludentă în contextul vanzarilor și ulterior al optimizării stocurilor, orizontul prezis este reprezentat de perioada *lead time* – timpul în care marfa este livrată de furnizor – în cazul prezentului proiect am ales 14 de zile, ci nu împărțire 80:20 (fig 11).

```
Missing values: 0
Time series length: 735

Train set: 721 observations
Test set: 14 observations
```

Fig. 11 Train Test Split

4.3. Modele de prognoză utilizate

ARIMA

Funcțiile ACF și PACF

³² <https://www.askpython.com/python/examples/split-data-training-and-testing-set>

Funcția de autocorelație (ACF) descrie relația dintre valorile unei serii temporale și valorile aceleiași serii deplasate în timp cu diferite lag-uri. ACF măsoară corelația totală dintre observații, incluzând atât efectele directe, cât și cele indirecte transmise prin lag-urile intermediare. Prin analiza graficului ACF se poate observa modul în care dependența dintre observații scade pe măsură ce crește distanța temporală dintre acestea, oferind indicii asupra structurii de dependență a seriei temporale.

Funcția de autocorelație parțială (PACF) măsoară relația dintre o serie temporală și valorile sale decalate, eliminând influența lag-urilor intermediare. Spre deosebire de ACF, care surprinde corelația cumulată, PACF izolează efectul direct al unui anumit lag asupra valorii curente a seriei. Această caracteristică permite identificarea relațiilor directe dintre observații separate de un anumit interval temporal.³³

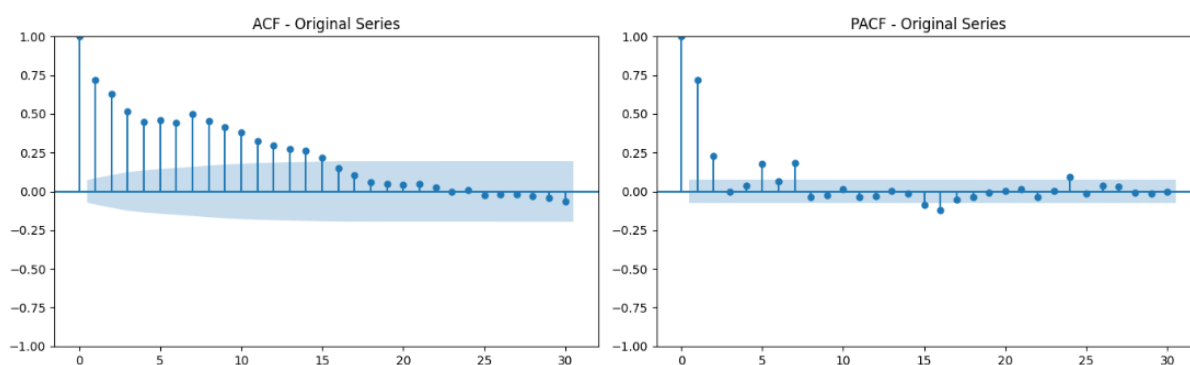


Fig. 12 ACF & PACF test pentru ARIMA

Analiza funcțiilor de autocorelație (ACF) și autocorelație parțială (PACF) – Fig 11 pentru seria zilnică evidențiază o dependență ridicată față de valorile recente, cu o autocorelație pozitivă care se diminuează gradual pe mai multe lag-uri. În PACF se observă lag-uri semnificative în principal la ordinul 1 și 2, iar ulterior valorile sunt, în majoritate, în interiorul intervalului de încredere, cu câteva deviații izolate. Acest tipar este compatibil cu un proces autoregresiv de ordin redus, sugerând un model ARIMA cu $p \approx 1-2$. În același timp, în intervalul de lag analizat (până la 30), ACF nu indică vârfuri periodice pronunțate, ceea ce sugerează absența unei sezonality scurte evidente; eventuale componente sezoniere pe orizonturi mai lungi necesită o analiză separată (de exemplu, ACF pe laguri mai mari sau decompoziție sezonieră).

³³ <https://towardsdatascience.com/interpreting-acf-and-pacf-plots-for-time-series-forecasting-af0d6db4061c/>

```

=====
Stationarity Tests for Original Training Series
=====

ADF Test (Null hypothesis: Series is non-stationary)
ADF Statistic: -5.126096
p-value: 0.000012
Critical Values:
 1%: -3.440
 5%: -2.866
10%: -2.569
✓ Series is STATIONARY (reject null hypothesis)

```

Fig. 13 ADF test

Staționaritatea seriei temporale a cantității vândute a fost evaluată utilizând testul Augmented Dickey-Fuller (fig 13). Ipoteza nulă a testului (H_0) presupune că seria este nestaționară (are unit root), iar ipoteza alternativă (H_1) presupune staționaritate. Rezultatele obținute indică o valoare a statisticii ADF de $-5,126$, cu $p\text{-value} = 0,000012$. Deoarece $p\text{-value}$ este semnificativ mai mic decât pragurile uzuale (de exemplu $0,05$), ipoteza nulă este respinsă. În plus, statistica ADF este mai mică decât valorile critice raportate la nivelurile de semnificație de 1% , 5% și 10% , ceea ce confirmă aceeași concluzie. Prin urmare, seria analizată poate fi considerată **staționară**, din perspectiva testului ADF, aspect relevant pentru aplicarea modelelor statistice care presupun staționaritatea datelor.

O serie temporală staționară este un tip de serie temporală ale cărei proprietăți statistice nu se modifică în timp. Aceasta înseamnă că comportamentul statistic al procesului rămâne constant, indiferent de momentul în care au fost înregistrate observațiile.³⁴ Seria are o **medie constantă pe termen lung** și nu are un "unit root" (nu crește sau scade la infinit, ca un trend liniar).

Tendențele care se repetă pe parcursul zilelor sau lunilor se numesc sezonaliitate în seriile temporale. Schimbările sezoniere, festivalurile și evenimentele culturale produc adesea aceste variații.³⁵

Nu facem diferențiere $\rightarrow d = 0$.

Model	P / "AR"	Q / "MA"	MAE	RMSE
ARIMA	5	4	6.4450	7.4818

Tabel 1. Rezultate ARIMA

Rezultatele obținute (tabel 1) pentru modelul ARIMA indică o capacitate limitată de prognoză, reflectată prin valori mai ridicate ale erorilor MAE și RMSE. Acest rezultat

³⁴ <https://www.tigerdata.com/learn/stationary-time-series-analysis>

³⁵ <https://www.analyticsvidhya.com/blog/2024/06/seasonality-in-time-series/>

sugerează că modelul reușește să surprindă nivelul mediu al cererii, însă nu explică în mod satisfăcător variațiile acesteia în timp. (Fig 12)

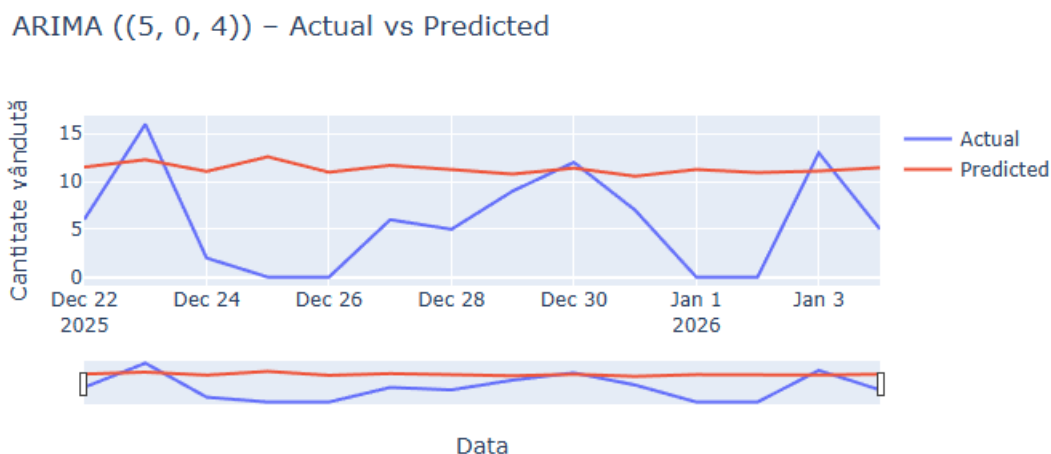


Fig. 14 Rezultat ARIMA

LSTM

Pentru prognoza cererii a fost utilizat un model de tip Long Short-Term Memory (LSTM), aparținând clasei rețelelor neuronale recurente, adecvat pentru modelarea seriilor temporale caracterizate prin dependențe temporale. Spre deosebire de modelele de tip machine learning clasic, în care observațiile sunt considerate independente, modelul LSTM permite captarea relațiilor dintre valorile succesive ale unei serii temporale.

Scalarea este, procesul de transformare a tuturor caracteristicilor dintr-un model mai aproape de un interval sau pe aceeași scară, de exemplu: toate valorile să fie între 0 și 1. De asemenea anumiți algoritmi, converg mai bine atunci când caracteristicile sunt scalate, deci performanța va fi îmbunătățită.

Având în vedere faptul că datele analizate sunt de tip serie temporală, setul de caracteristici utilizat în modelare nu este compus preponderent din variabile independente distincte, ci din aceeași variabilă principală, respectiv cantitatea vândută, deplasată în timp sub forma unor ferestre temporale. Astfel, scalarea a fost aplicată pentru a asigura consistența numerică a valorilor istorice utilizate în procesul de învățare al modelului.

Am folosit scalarea min-max (MinMaxScaler) – denumită și normalizare – ce transformă datele astfel încât fiecare valoare să se încadreze între 0 și 1, ce funcționează conform formulei din Ecuația 1. (Eq. 1).³⁶

³⁶ <https://towardsdatascience.com/data-scaling-101-standardization-and-min-max-scaling-explained-60789833e160/>

$$X(\text{scaled}) = (X_i - X_{\min}) / (X_{\max} - X_{\min})$$

Eq. 1 Ecuatia de normalizare pentru un punct de date X_i

În cadrul acestui studiu, predicția cantității vândute la momentul t_{se} realizează pe baza unei ferestre temporale de observații anterioare, denumită *lookback window*. Aceasta reprezintă numărul de pași temporali din trecut utilizați ca intrare în model. În implementarea curentă, dimensiunea ferestrei a fost stabilită la 14 zile, valoare aleasă pentru a surprinde atât variațiile pe termen scurt, cât și eventualele pattern-uri săptămânale ale cererii.

Rolul datasetului istoric extins

Deși setul de date analizat acoperă o perioadă de aproximativ doi ani, acesta nu este utilizat integral pentru o singură predicție. Istoricul extins servește la generarea unui număr mare de secvențe de antrenare, fiecare secvență fiind construită prin deplasarea ferestrei temporale de tip *lookback* de-a lungul seriei. Acest mecanism permite modelului să învețe tipare recurente ale cererii în contexte temporale diferite, menținând în același timp o structură de predicție realistă din punct de vedere operațional.

Rezultat:

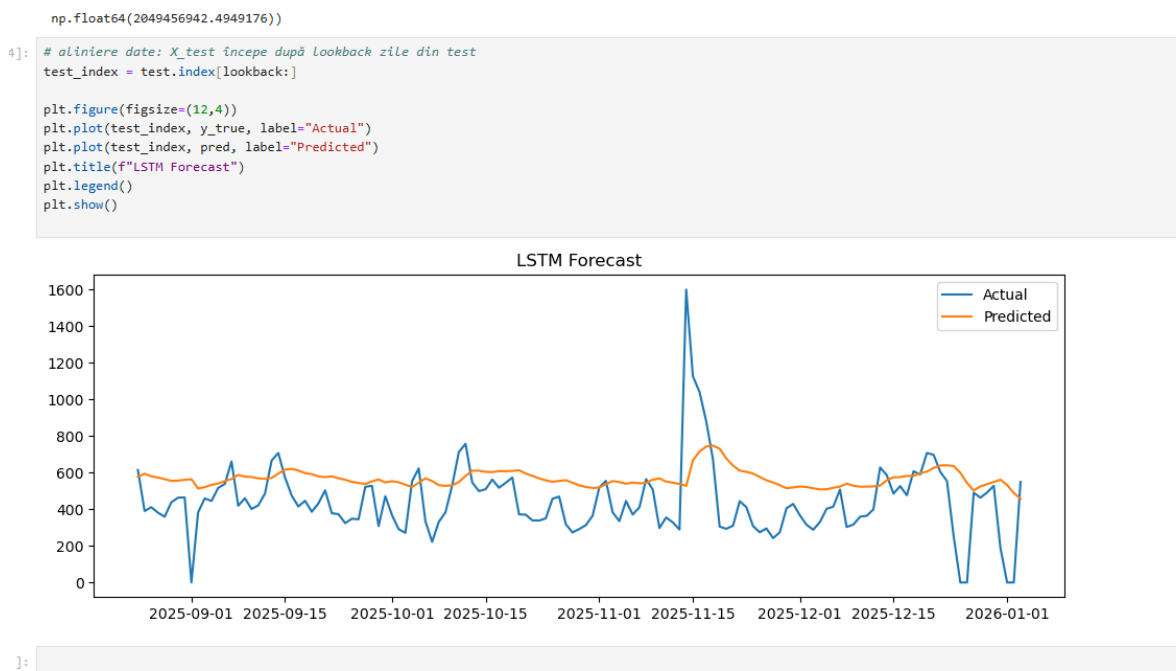


Fig. 15 LSTM Predicted

Pe scurt, rezultatul cuprinde valorile medii prezise, valori stabile, utilizabile.

Graficul comparativ dintre valorile reale și cele estimate de modelul LSTM evidențiază capacitatea acestuia de a surprinde nivelul general și tendința cererii, reducând în același timp impactul fluctuațiilor extreme izolate. Predicțiile obținute sunt mai netede decât seria observată, ceea ce indică faptul că modelul filtrează zgomotul și evenimentele atipice, concentrându-se pe comportamentul recurent al cererii. Această caracteristică este avantajoasă în contextul planificării stocurilor, unde estimarea cererii medii este mai relevantă decât replicarea exactă a variațiilor zilnice extreme.

```
[63]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_true, pred)

mse = mean_squared_error(y_true, pred)
rmse = np.sqrt(mse)

mape = np.mean(np.abs((y_true - pred) / np.maximum(y_true, 1))) * 100

mae = float(mae)
rmse = float(rmse)
mape = float(mape)

mae, rmse, mape
```

```
[63]: (5.25101833873325, 8.066700162880037, 83.71565125019796)
```

```
[64]: from sklearn.metrics import r2_score

r2 = r2_score(y_true, pred)
r2
```

```
[64]: 0.5388633412727699
```

```
[65]: import numpy as np

r = np.corrcoef(y_true, pred)[0, 1]
float(r)
# arata daca modelul urmareste trandul, nu cat de exact prezice
```

```
[65]: 0.7346987072878048
```

Analiza performanței modelului LSTM (lookback = 21)

Modelul LSTM multivariant, construit pe baza unei ferestre temporale de 21 de zile, a fost evaluat utilizând setul de testare, prin intermediul unor metrice specifice problemelor de regresie aplicate seriilor temporale.

Valoarea MAE, de 5,25 unități, arată că, în medie, diferența dintre cantitatea estimată de model și cea observată în realitate este de aproximativ cinci unități pe zi. Raportat la

nivelul general al cererii și la variabilitatea caracteristică datelor din retail, această eroare poate fi considerată rezonabilă, indicând o acuratețe suficientă pentru utilizarea modelului în procese de planificare a stocurilor.

În cazul **RMSE**, valoarea obținută este de **8,07 unități**, mai ridicată decât MAE, ceea ce evidențiază faptul că modelul înregistrează erori mai mari în situațiile în care apar variații extreme ale cererii. Acest rezultat este așteptat în contextul seriilor temporale provenite din sisteme ERP, unde pot apărea vârfuri izolate sau fluctuații bruște, pe care modelul LSTM tinde să nu le reproducă exact.

Coeficientul de determinare R^2 , cu valoarea de **0,54**, indică faptul că modelul explică aproximativ jumătate din variația cantității vândute. Pentru o serie temporală reală, influențată de factori externi și caracterizată printr-un nivel ridicat de zgomot, acest rezultat sugerează o capacitate bună de surprindere a comportamentului general al cererii. De asemenea, **coeficientul de corelație Pearson**, egal cu **0,73**, confirmă existența unei relații pozitive puternice între valorile reale și cele estimate. Acest lucru arată că modelul reușește să urmărească direcția și evoluția generală a cererii, chiar dacă nu redă cu precizie toate variațiile zilnice.

În ceea ce privește **MAPE**, valoarea ridicată obținută (**83,7%**) este influențată de prezența valorilor nule sau foarte mici în seria analizată, situație care conduce la supraestimarea erorii procentuale. Din acest motiv, această metrică nu a fost considerată relevantă pentru evaluarea principală a performanței modelului. ³⁷

Concluzie

În ansamblu, modelul LSTM multivariant cu o fereastră de 21 de zile oferă rezultate satisfăcătoare pentru estimarea nivelului general al cererii și pentru identificarea tendințelor dominante. Deși nu surprinde cu exactitate fluctuațiile extreme sau evenimentele izolate, caracterul său stabil și conservator îl face adecvat pentru susținerea deciziilor operaționale legate de gestiunea și optimizarea stocurilor.

LSTM 14

Model	Lookback (zile)	MAE	RMSE	R^2	r (Pearson)
LSTM	14	5,16	8,20	0,50	0,71
LSTM	21	5,25	8,07	0,54	0,73

³⁷ <https://www.geeksforgeeks.org/machine-learning/regression-metrics/>

Tabel 1. Compararea performanței modelului LSTM în funcție de dimensiunea ferestrei temporale

Deși modelul LSTM cu o fereastră temporală de 14 zile obține o eroare medie absolută ușor mai redusă, diferența față de modelul cu lookback de 21 de zile este nesemnificativă. În schimb, modelul cu 21 de zile înregistrează valori mai bune pentru RMSE, coeficientul de determinare și corelația Pearson, indicând o capacitate superioară de a surprinde structura temporală și tendința generală a cererii. Prin urmare, modelul LSTM cu lookback de 21 de zile a fost ales ca variantă finală pentru analiza ulterioară.

XGBoost

Feature Engineering

În XGBoost, feature engineering (Fig. 16) este esențial pentru prognoza de serie temporală, deoarece modelul nu „știe” automat ordinea timpului. Practic, transformăm seria într-un set de variabile explicative (X) care descriu calendarul și istoricul recent al cererii (cantității), iar ținta (y) rămâne CANTITATE la data curentă.

În primul rând, din coloana de tip dată (DATA) au fost extrase variabile calendaristice precum ziua din săptămână (DOW, codificată 0–6), luna (month) și ziua din lună (ZI_DIN_LUNA). Acestea permit modelului să surprindă eventuale diferențe sistematice între zile (de exemplu, între zile lucrătoare și weekend) și variații sezoniere pe parcursul lunilor. Complementar, a fost introdusă o variabilă binară (ESTE_WEEKEND) care marchează zilele de sâmbătă și duminică, cu rolul de a concentra într-un singur semnal efectul „weekend” asupra vânzărilor.

Pentru a integra dependențele temporale specifice seriei, au fost generate variabile de tip întârziere (lag features), prin deplasarea seriei țintă (CANTITATE) cu 1, 7, 14 și 21 de zile. Astfel, lag_1 reflectă inerția de la o zi la alta, iar lag_7, lag_14 și lag_21 oferă informații despre repetabilitatea săptămânală și despre cicluri pe termen scurt (două–trei săptămâni), frecvent întâlnite în datele de retail. În plus, pentru a obține o estimare a nivelului recent al cererii și a variației acesteia, au fost calculate statistici pe ferestre mobile (rolling), precum media pe 7 și 14 zile (roll_mean_7, roll_mean_14) și deviația standard pe 7 zile (roll_std_7). Acestea descriu atât tendința locală a cererii, cât și volatilitatea recentă, oferind modelului un context suplimentar față de valorile punctuale (lag-uri).

Un aspect important al construcției acestor variabile a fost evitarea scurgerii de informație (data leakage). Din acest motiv, statisticile de tip rolling au fost calculate pe valori deplasate cu o zi (shift(1)), astfel încât fereastra de agregare să includă exclusiv observații anterioare momentului prezis. În acest mod, la momentul t , modelul utilizează doar informațiile care ar fi fost disponibile în mod realist până la $t-1$. Feature engineering-ul descris asigură introducerea explicită a structurii temporale în date și permite utilizarea XGBoost într-un cadru de prognoză, în care variabilele explicative reprezintă atât calendarul, cât și istoricul cererii pe intervale relevante.

Lag = "copie" din trecut (valoare exactă de acum k zile).

Rolling = statistică (medie / std / min / max etc.) pe o fereastră de valori trecute.

```
df_feat["DOW"] = df_feat["DATA"].dt.dayofweek          # 0..6
df_feat["month"] = df_feat["DATA"].dt.month
df_feat["ESTE_WEEKEND"] = (df_feat["DOW"] >= 5).astype(int)

# dacă ai deja LUNA ca yyyy-mm, păstrează-o ca perioadă (opțional)
# df_feat["YYYY_MM"] = df_feat["DATA"].dt.to_period("M").astype(str)

# opțional: zi din lună (uneori surprinde patternuri de început/sfârșit)
df_feat["ZI_DIN_LUNA"] = df_feat["DATA"].dt.day

# lag-uri din target
for lag in [1, 7, 14, 21]:
    df_feat[f"lag_{lag}"] = df_feat["CANTITATE"].shift(lag)

# rolling (folosește shift(1) ca să eviți leakage!)
df_feat["roll_mean_7"] = df_feat["CANTITATE"].shift(1).rolling(7).mean()
df_feat["roll_mean_14"] = df_feat["CANTITATE"].shift(1).rolling(14).mean()
df_feat["roll_std_7"] = df_feat["CANTITATE"].shift(1).rolling(7).std()

df_feat
```

Fig. 16 XGBoost feature engineering

Pentru a preveni data leakage, în setul de date utilizat au fost eliminate variabile care pot incorpora informație derivată din vânzări/ieșiri realizate în aceeași zi sau din efecte generate de acestea.

Data leakage (scurgere de informație) reprezintă situația în care, în etapa de antrenare sau evaluare, modelul primește ca variabile de intrare informații care nu ar fi disponibile în mod realist la momentul efectuării prognozei.³⁸

Antrenarea și rularea modelului, generarea predicțiilor :

³⁸ <https://www.kaggle.com/code/alexisbcook/data-leakage>

Explicatie XGBRegressor + parametri:

XGBRegressor(...) – inițializează un model XGBoost pentru regresie, bazat pe **gradient boosting** cu arbori de decizie. Modelul construiește secvențial mai mulți arbori, fiecare contribuind la corectarea erorilor făcute de ansamblul anterior.

n_estimators=600 – reprezintă **numărul de arbori** (runde de boosting) din ansamblu. O valoare mai mare permite modelului să învețe tipare mai complexe, însă poate crește timpul de antrenare și riscul de supraînvățare dacă nu este controlat prin regularizare și rata de învățare.

max_depth=6 – stabilește **adâncimea maximă** a fiecărui arbore. Adâncimi mai mari permit reguli mai detaliate (captarea interacțiunilor dintre variabile), dar pot conduce la supraînvățare; adâncimi moderate sunt utilizate pentru echilibru între flexibilitate și generalizare.

learning_rate=0.03 – controlează **mărimea contribuției** fiecărui arbore nou (shrinkage). O rată mai mică face învățarea mai graduală și, de regulă, mai stabilă, necesitând însă un număr mai mare de arbori pentru performanță bună.

subsample=0.9 – indică proporția de **observații (rânduri)** selectate aleator pentru antrenarea fiecărui arbore. Sub-eșantionarea reduce corelația dintre arbori și acționează ca mecanism de regularizare, diminuând riscul de supraînvățare.

colsample_bytree=0.9 – indică proporția de **variabile (feature-uri)** selectate aleator pentru fiecare arbore. Această selecție aleatoare crește robustețea modelului și reduce dependența de un număr mic de predictorii dominanți.

reg_lambda=1.0 – parametrul de **regularizare L2** (penalizare) aplicat greutăților modelului. Regularizarea L2 limitează complexitatea soluției și contribuie la îmbunătățirea capacității de generalizare pe date noi.

random_state=42 – fixează sămânța generatorului aleator pentru procedurile care implică randomizare (de exemplu, subsample și colsample_bytree), asigurând **reproductibilitatea** rezultatelor între rulări.³⁹

³⁹ <https://xgboost.readthedocs.io/en/stable/parameter.html>

Model	Lookback (max lag zile)	MAE	RMSE	R ²
XGBoost	21	0,0189	0,0189	0,0189

Tabel 2. Metrice rezultate XGBoost

Metrice rezultate (Tabel 2.) :

MAE = 0.0189 – eroarea medie absolută: arată cu cât diferă, în medie, predicțiile XGBoost față de valorile reale (în aceleași unități ca ținta). Este relevantă în prognoza seriilor temporale deoarece se interpretează direct și nu penalizează excesiv erorile rare foarte mari.

RMSE = 0.0239 – rădăcina erorii medii pătratice: penalizează mai puternic abaterile mari decât MAE. Este relevantă pentru time series când erorile mari (vârfuri ratate) sunt costisitoare operațional.

R² = 0.99997 – coeficientul de determinare: indică o potrivire aproape perfectă pe setul de test, însă în prognoza seriilor temporale se folosește mai ales ca indicator complementar, deoarece poate fi influențat de variația redusă a seriei sau de o evaluare prea optimistă dacă split-ul/feature engineering-ul nu respectă strict ordinea temporală.

4.5. Evaluarea performanței modelelor

- Metrice utilizate
- Compararea rezultatelor

4.6. Aplicația de tip dashboard pentru vizualizare

Python Streamlit

Aplicația este dezvoltată utilizând framework-ul Streamlit, care pornește un server web local pe stația de lucru, expunând aplicația prin intermediul unui port TCP dedicat (implicit

portul 8501), fiind accesibilă fie direct de pe stația pe care rulează aplicația (<http://localhost:8501>) , fie prin intermediul adresei IP din rețeaua internă (Network URL), de exemplu <http://192.168.1.128:8501>.

Acest mod de acces este specific mediilor organizaționale care utilizează infrastructuri proprii de tip *on-premise* sau centre de date interne, în care resursele informatice sunt partajate în cadrul unei rețele locale securizate (LAN), fără expunerea aplicației în mediul public (Internet).

Accesarea aplicației este limitată la utilizatorii aflați în aceeași rețea internă, ceea ce permite controlul asupra securității datelor și respectarea politicilor interne de confidențialitate, fiind o abordare frecvent utilizată în cadrul organizațiilor care gestionează date sensibile.

Aplicația a fost dezvoltată utilizând un **mediu virtual Python (Python Virtual Environment)**, ales pentru a asigura un control riguros asupra dependențelor software utilizate în cadrul proiectului. Acest mediu izolat permite rularea aplicației în condiții identice pe sisteme diferite, prin utilizarea acelorași versiuni ale bibliotecilor necesare, conform specificațiilor din fișierul *requirements.txt*.

Astfel, aplicația poate fi rulată în aceleași condiții software pe orice sistem, fără diferențe de versiuni ale pachetelor utilizate. mecanism oferă **izolație față de alte proiecte Python** existente pe sistemul de operare, prevenind conflictele între biblioteci sau modificările neintenționate ale mediului global. ⁴⁰

- Descriere generală a aplicației

Pentru dezvoltarea aplicației software și refactorizarea codului Python au fost utilizate instrumente moderne de asistare a programării, precum **GitHub Copilot** (integrat în Visual Studio Code) și platforma **Replit**, care au sprijinit procesul de generare și organizare a codului.

Logica de analiză a datelor, definirea modelelor predictive și interpretarea rezultatelor au fost realizate și validate de autor.

⁴⁰ <https://docs.python.org/3/library/venv.html>

Capitolul 5. Discuții și analize

5.1. Interpretarea rezultatelor

- Impact asupra gestiunii stocurilor

5.2. Implicații pentru compania analizată

- Decizii de aprovizionare
- Optimizarea stocului tampon

5.3. Implicații pentru domeniul sistemelor informatice integrate

CAPITOLUL 6 – CONCLUZII, PROPUNERI ȘI IMPLICAȚII PRACTICE

Limitări, concluzii și recomandări practice

6.1. Limitările studiului

Un prim set de limitări este determinat de **caracteristicile datelor utilizate**. Analiza s-a bazat pe o serie temporală zilnică, în care pot apărea **vârfuri izolate (outlieri)** și variații operaționale (promoții, livrări, rupturi de stoc), care nu reflectă întotdeauna un

comportament stabil al cererii. În plus, în situațiile în care lipsesc observații, completarea lor (de exemplu prin interpolare sau înlocuire cu zero) poate influența rezultatele și interpretarea componentelor de trend/sezonalitate și a performanței modelelor.

O limitare metodologică importantă este legată de **orizontul de predicție** și modul de estimare multi-step. În practică, necesarul de prognoză depinde de lead time și de politica de aprovizionare; dacă se dorește prognoză pe mai multe zile înainte, pot apărea erori cumulative (în special la abordările recursive) și este necesară o validare suplimentară pe orizonturi diferite.

În final, lucrarea a urmărit validarea modelelor din perspectivă predictivă, însă **nu a inclus o optimizare completă a deciziilor de stoc** (de exemplu calculul formal al stocului de siguranță, costuri de holding, costuri de ruptură de stoc și constrângeri de capacitate). Astfel, rezultatele trebuie interpretate ca suport pentru decizie, nu ca soluție completă de optimizare.

6.2. Concluziile principale

Lucrarea a urmărit analiza și evaluarea posibilității de integrare a prognozei cererii, realizată prin metode statistice și de machine learning, într-un cadru aplicabil pentru gestiunea stocurilor în retail. Din perspectiva obiectivelor formulate, concluziile pot fi sintetizate astfel:

(1) Prognoza cererii ca instrument pentru îmbunătățirea planificării comenzilor
Rezultatele indică faptul că introducerea prognozei bazate pe date istorice permite o planificare mai informată comparativ cu abordările simplificate bazate pe medii istorice. Analizele au evidențiat existența unor tipare temporale recurente (în special la nivel săptămânal), precum și apariția unor vârfuri izolate care nu reflectă un trend de lungă durată. Prin urmare, un model predictiv poate sprijini planificarea, deoarece captează dependențele pe termen scurt și componentele calendaristice, reducând riscul ca deciziile să se bazeze exclusiv pe medii globale care „aplatizează” variația și pot ignora sezonalitatea.

În special, utilizarea XGBoost cu feature engineering specific seriilor temporale (lag-uri și statistici rolling) demonstrează că algoritmi de tip boosting pot modela relații neliniare între istoricul recent și cererea curentă. În același timp, rezultatele LSTM arată că rețelele neuronale pot surprinde dependențe temporale, însă necesită un proces mai complex de pregătire a datelor și un control atent al hiperparametrilor și al

ferestrei de intrare. Astfel, valoarea practică a prognozei depinde nu doar de acuratețe, ci și de costul implementării și întreținerii modelului.

(2) Integrarea fluxului automatizat în Python într-un sistem ERP

Integrarea unui flux automatizat de prognoză este posibilă dacă este proiectată modular și non-intruziv față de ERP. În cadrul proiectului, logica de prognoză este separată în pași clari: extragerea datelor (din surse ERP/raportare), preprocesare, generarea feature-urilor, rularea modelului și publicarea rezultatelor într-o interfață utilizabilă. O astfel de arhitectură reduce riscurile asupra stabilității operaționale, întrucât componenta de prognoză funcționează ca un serviciu de analiză, fără a modifica sistemul tranzacțional.

Un element esențial pentru implementare este reproductibilitatea mediului de execuție. În context organizațional, diferențele de versiuni ale bibliotecilor sau setări locale pot conduce la rezultate inconsistent. De aceea, încapsularea aplicației (de exemplu cu Docker) reprezintă o soluție practică pentru a asigura aceleași dependențe și același comportament al aplicației între medii diferite (dezvoltare, test, producție). În plus, separarea fluxului de prognoză de ERP permite scalare și mentenanță independentă.

(3) Care este diferența dintre abordarea tradițională (vânzare medie zilnică) și abordarea predictivă în privința nivelurilor de stoc rezultate?

Abordarea bazată pe medie are avantajul simplității, însă reacționează lent la schimbări și poate ignora sezonabilitatea pe zile/săptămâni. Abordarea predictivă oferă un mecanism mai flexibil, capabil să includă informație calendaristică și istoricul recent, ceea ce poate conduce la propuneri de reprovizionare mai bine aliniate cu dinamica cererii. Diferența devine relevantă mai ales când cererea are modele recurente (ex. săptămânal) și când există variații care nu pot fi captate de o medie globală.

6.3. Recomandări practice

- Pentru companie

În locul utilizării unui mediu virtual Python configurat local, aplicația poate fi încapsulată sub forma unui container Docker, care include atât codul aplicației, cât și toate dependențele necesare. Această abordare permite partajarea aplicației și rularea sa într-un mod consistent de către membrii echipei interne a organizației, indiferent de mediul local de execuție.

Validare continuă și baseline: În exploatare, performanța modelului trebuie urmărită periodic (MAE/RMSE) și comparată cu un baseline temporal (ex. „valoarea de ieri” sau „valoarea de acum 7 zile”), pentru a confirma că beneficiul este real și stabil în timp.

Tratamentul evenimentelor și outlierilor: Vârfurile izolate trebuie tratate explicit (reguli de detecție/anotare promoții, lipsă stoc, schimbări de preț), deoarece pot distorsiona prognoza și pot duce la supra-aprovizionare.

Alegerea orizontului după lead time: Orizontul de prognoză trebuie corelat cu lead time-ul furnizorilor și cu frecvența de comandă (de ex. 7–14 zile), pentru ca rezultatul să fie utilizabil în decizia de reprovizionare.

BIBLIOGRAFIE

StartCo - Newsletter pentru antreprenori

<https://www.oracle.com/erp/what-is-erp/>

SAP, 2024 - <https://www.sap.com/products/erp/what-is-erp.html>

<https://www.eazystock.com/blog/erp-inventory-management-how-advanced-is-your-erp/>

<https://www.ibm.com/think/topics/ai-demand-forecasting>

<https://throughput.world/blog/ai-in-retail-supply-chain/>

<https://www.ibm.com/think/topics/machine-learning>

<https://www.geeksforgeeks.org/machine-learning/time-series-forecasting-as-supervised-learning/>

Unsupervised Learning in Supply Chain Demand - growth-onomics

<https://doi.org/10.1016/j.tre.2020.101926>

Hyndman & Athanasopoulos, 2021, Capitolul 3 – "Time series decomposition"

<https://otexts.com/fpp3/decomposition.html>

<https://otexts.com/fpp3/tspatterns.html>

<https://www.sciencedirect.com/science/article/abs/pii/S0169207004000792?via%3Dihub>

Statistical Learning vs. Machine Learning: What's the Difference? | Coursera

<https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html>

<https://otexts.com/fpp2/arima.html>

<https://otexts.com/fpp2/seasonal-arima.html>

<https://www.statsmodels.org/stable/generated/statsmodels.tsa.holtwinters.ExponentialSmoothing.html>

<https://xgboost.readthedocs.io/en/stable/>

<https://www.analyticsvidhya.com/blog/2024/01/xgboost-for-time-series-forecasting/>

[https://scikit-](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html)

[learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html)
(gol)

Brownlee, J. (2017). An Introduction to Long Short-Term Memory Networks (LSTM).

Towards Data Science. <https://towardsdatascience.com/an-introduction-to-long-short-term-memory-networks-lstm-27af36dde85d/>

https://facebook.github.io/prophet/docs/quick_start.html#python-api

https://facebook.github.io/prophet/docs/seasonality%2C_holiday_effects%2C_and_r

[egressors.html?utm_source=chatgpt.com](#)

https://dev.to/mondal_sabbha/understanding-mae-mse-and-rmse-key-metrics-in-machine-learning-4la2

<https://www.ijsat.org/papers/2025/1/2644.pdf>

<https://procurementmag.com/articles/schwarz-it-better-inventory-management-through-ai>

<https://www.slimstock.com/ro/blog/prognoza-cererii/>

Pareto principle - Wikipedia

<https://www.geeksforgeeks.org/python/a-beginners-guide-to-streamlit/>

<https://www.askpython.com/python/examples/split-data-training-and-testing-set>

<https://towardsdatascience.com/interpreting-acf-and-pacf-plots-for-time-series-forecasting-af0d6db4061c/>

<https://www.tigerdata.com/learn/stationary-time-series-analysis>

<https://www.analyticsvidhya.com/blog/2024/06/seasonality-in-time-series/>

<https://www.askpython.com/python/examples/split-data-training-and-testing-set>

<https://towardsdatascience.com/data-scaling-101-standardization-and-min-max-scaling-explained-60789833e160/>

<https://www.geeksforgeeks.org/machine-learning/regression-metrics/>

<https://www.kaggle.com/code/alexisbcook/data-leakage>

<https://xgboost.readthedocs.io/en/stable/parameter.html>

<https://docs.python.org/3/library/venv.html>

<https://www.geeksforgeeks.org/python/libraries-in-python/>